THE SILENT INVIGILATOR
AUTONOMOUS REAL-TIME EXAM MALPRACTICE DETECTION SYSTEM

A MINI PROJECT REPORT

by

BENEDICT CHACKO MATHEW (VJC23CS062)
DAVOOD JAMAL (VJC23CS069)
JUDITH K VIJESH (VJC23CS117)
MUHAMMED JAMALUDHEEN UC (VJC23CS142)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
VISWAJYOTHI COLLEGE OF ENGINEERING AND TECHNOLOGY, VAZHAKULAM

MAY 2026

THE SILENT INVIGILATOR
AUTONOMOUS REAL-TIME EXAM MALPRACTICE DETECTION SYSTEM

A MINI PROJECT REPORT

by

BENEDICT CHACKO MATHEW (VJC23CS062)
DAVOOD JAMAL (VJC23CS069)
JUDITH K VIJESH (VJC23CS117)
MUHAMMED JAMALUDHEEN UC (VJC23CS142)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

under the guidance

of

Ms. ASHLY GEORGE

Assistant Professor, CSE Department

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
VISWAJYOTHI COLLEGE OF ENGINEERING AND TECHNOLOGY, VAZHAKULAM

MAY 2026

VISWAJYOTHI COLLEGE OF ENGINEERING AND TECHNOLOGY, VAZHAKULAM

Department of Computer Science and Engineering

Vision

Moulding socially responsible and professionally competent Computer Engineers to adapt to the dynamic technological landscape

Mission

1. Foster the principles and practices of computer science to empower life-long learning and build careers in software and hardware development.

2. Impart value education to elevate students to be successful, ethical and effective problem-solvers to serve the needs of the industry, government, society and the scientific community.

3. Promote industry interaction to pursue new technologies in Computer Science and provide excellent infrastructure to engage faculty and students in scholarly research activities.

Program Educational Objectives

Our Graduates

1. Shall have creative aid critical reasoning skills to solve technical problems ethically and responsibly to serve the society.

2. Shall have competency to collaborate as a team member and team leader to address social, technical and engineering challenges.

3. Shall have ability to contribute to the development of the next generation of information technology either through innovative research or through practice in a corporate setting

4. Shall have potential to build start-up companies with the foundations, knowledge and experience they acquired from undergraduate education

Program Outcomes

1. **Engineering knowledge**:Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. **Problem analysis**:Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences

3. **Design / development of solutions**:Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety and the cultural, societal and environmental considerations.

4. **Conduct investigations of complex problems**:Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data and synthesis of the information to provide valid conclusions.

5. **Modern tool usage**:Create, select and apply appropriate techniques, resources and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

6. **The engineer and society:**Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. **Environment and sustainability:**Understand the impact of the professional engineering solutions in societal and environmental contexts and demonstrate the knowledge of and need for sustainable development.

8. **Ethics:**Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice

9. **Individual and team work:**Function effectively as an individual and as a member or leader in diverse teams and in multidisciplinary settings

10. **Communication:**Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations and give and receive clear instructions.

11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and unread in a team, to manage projects and in multidisciplinary environments.

12. **Life-long learning**:Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes

1. Ability to integrate theory and practice to construct software systems of varying complexity
2. Able to Apply Computer Science skills, tools and mathematical techniques to analyse, design and model complex systems
3. Ability to design and manage small-scale projects to develop a career in a related industry.

VISWAJYOTHI COLLEGE OF ENGINEERING AND TECHNOLOGY, VAZHAKULAM

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that the mini project work entitled **"THE SILENT INVIGILATOR AUTONOMOUS REAL-TIME EXAM MALPRACTICE DETECTION SYSTEM"** is a bonafide work done by **BENEDICT CHACKO MATHEW (VJC23CS062),DAVOOD JAMAL (VJC23CS069),JUDITH K VIJESH (VJC23CS117) and MUHAMMED JAMALUDHEEN UC (VJC23CS142)** in partial fulfillment of the award of the Degree of **Bachelor of Technology in Computer Science & Engineering** from APJ Abdul Kalam Technological University, Thiruvananthapuram, Kerala during the academic year 2025-2026

Internal Supervisor **External Supervisor**

Mini Project Coordinator **Head of Department**

ACKNOWLEDGEMENT

First and foremost, We thank **God Almighty** for His divine grace and blessings in making all these possible. May He continue to lead us in the years. It is our privilege to render our heartfelt thanks to our most beloved Manager **Msg. Dr. PIUS MALEKANDATHIL**, our Director **Rev. Fr. PAUL PARATHAZHAM** and our Principal **Dr. K K RAJAN** for providing us the opportunity to do this mini project during the sixth semester of our B.Tech degree course. We are deeply thankful to our Head of the Department, **Dr. AMEL AUSTINE** for his support and encouragement. We would like to express our sincere gratitude and heartfelt thanks to our Mini Project Guide **Ms. ASHLY GEORGE**, Asst. Professor, Department of Computer Science and Engineering for her motivation, assistance and help for the project. We also express our sincere thanks to the Mini Project Coordinator **Ms. ELIZABETH ANNS**, Asst. Professor, Department of Computer Science and Engineering for her guidance and support. We also thank all the staff members of the Computer Science Department for providing their assistance and support. Last, but not the least, We thank all our friends and family for their valuable feedback from time to time as well as their help and encouragement.

DECLARATION

We undersigned hereby declare that the mini project report **"THE SILENT INVIGILATOR AUTONOMOUS REAL-TIME EXAM MALPRACTICE DETECTION SYSTEM"**, submitted for partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology of the APJ Abdul Kalam Technological University is a bonafide work done under the supervision of **Ms. Ashly George**. This submission represents ideas in our own words and where ideas or words of others have been included, We have adequately and accurately cited and referenced the original sources. We also declare that We have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or formed the basis for the award of any degree, diploma or similar title of any other University.

BENEDICT CHACKO MATHEW
DAVOOD JAMAL
JUDITH K VIJESH
MUHAMMED JAMALUDHEEN UC

Place : Vazhakulam

ABSTRACT

Examination malpractice affects the fairness, credibility, and integrity of academic evaluation, while conventional manual invigilation can be limited by human fatigue, delayed observation, and restricted monitoring coverage. To address these challenges, this project implements **The Silent Invigilator**, an autonomous real-time exam malpractice detection system that uses computer vision and deep learning to assist surveillance in examination environments. The system performs 3D head pose estimation using solvePnP with Rodrigues transforms, eye gaze tracking through iris ratio analysis, and mobile phone detection using YOLOv8 Nano with SAHI (Sliced Aided Hyper Inference). It integrates MediaPipe for facial landmark detection, ByteTrack for multi-person tracking, and OpenCV for real-time image processing. A risk scoring algorithm combines head pose deviation, gaze aversion, mouth activity, and detected prohibited objects to generate severity-based alerts. The backend uses Flask-SocketIO for real-time communication, SQLite for persistent logging, and JWT-based authentication for role-based access control. The system helps reduce dependence on manual invigilation, minimize human bias, and maintain a tamper-resistant audit trail, while supporting desktop dashboards and mobile client interfaces through MJPEG streaming.

Key Words: Computer Vision, Deep Learning, YOLOv8, MediaPipe, Real-time Detection, Head Pose Estimation, Gaze Tracking, Risk Scoring, Flask-SocketIO, SQLite, Role-based Access Control

Contents

List of Figures

List of Tables

List of Abbreviations

AI	Artificial Intelligence
ML	Machine Learning
CV	Computer Vision
CNN	Convolutional Neural Network
REST	Representational State Transfer
JWT	JSON Web Token
YOLO	You Only Look Once
IoU	Intersection over Union
FPS	Frames Per Second
MAR	Mouth Aspect Ratio
MJPEG	Motion JPEG
API	Application Programming Interface
JSON	JavaScript Object Notation
UML	Unified Modeling Language
DFD	Data Flow Diagram
SAHI	Sliced Aided Hyper Inference

Chapter 1

INTRODUCTION

Examination integrity is a cornerstone of educational excellence and institutional credibility. Yet traditional examination invigilation systems rely heavily on manual surveillance by human invigilators, introducing significant limitations: human fatigue reduces vigilance over extended examination periods, inherent biases may lead to inconsistent enforcement of examination rules, and maintaining adequate invigilation staff creates substantial logistical and financial burdens for educational institutions. Moreover, student malpractice behaviors continue to evolve, with increasing sophistication in using unauthorized technology and coordinating dishonest practices, often outpacing the detection capabilities of human supervisors.

Recent advancements in computer vision and artificial intelligence offer promising solutions to these challenges. Deep learning models can process visual information continuously without fatigue, apply consistent rule enforcement without subjective bias, and generate comprehensive audit trails for accountability. The integration of multiple behavioral analysis modalities—including head pose estimation, gaze tracking, hand gesture recognition, and object detection—provides a holistic assessment of student conduct that exceeds human capability.

The Silent Invigilator project addresses these challenges by developing an autonomous, real-time exam malpractice detection system that combines cutting-edge techniques from computer vision, deep learning, and real-time signal processing. The system provides objective, data-driven surveillance that augments or replaces manual invigilation, ensuring examination integrity while minimizing institutional overhead and human intervention.

1.1 Problem Definition

The traditional invigilation process in educational institutions faces multifaceted challenges:

1. **Human Limitations**: Invigilators experience fatigue during extended examination periods, reducing vigilance and increasing the likelihood of missing malpractice instances.
2. **Subjective Bias**: Manual surveillance introduces inconsistencies in rule enforcement, as

different invigilators may interpret student behavior differently or enforce penalties inconsistently.

3. **Resource Constraints**: Educational institutions must allocate significant financial and human resources to recruit, train, and schedule invigilators across multiple examination centers simultaneously.

4. **Evolving Threats**: Students employ increasingly sophisticated malpractice techniques, including concealed mobile phones, covert communication systems, and coordinated cheating strategies that are difficult for humans to detect reliably.

5. **Accountability Gaps**: Traditional invigilation lacks comprehensive logging and audit trails, making it difficult to provide objective evidence of malpractice or defend institutional decisions in case of disputes.

6. **Scalability Issues**: Expanding examination coverage to remote or rural examination centers becomes economically unfeasible with manual invigilation, limiting access to fair and consistent examination conditions.

These challenges collectively create a pressing need for an automated, objective, and scalable examination invigilation system that can operate continuously, consistently, and without human fatigue or bias.

1.2 Objective

The primary objectives of The Silent Invigilator project are:

1. **Autonomous Detection**: Develop an AI-driven system capable of detecting malpractice behaviors in real-time without human intervention, utilizing multiple behavioral and object detection modalities.

2. **Objective Severity Scoring**: Implement a comprehensive risk-scoring algorithm that aggregates multiple indicators (head pose, gaze direction, mouth activity, object presence) into a single, quantifiable severity metric, eliminating subjective interpretation.

3. **Real-time Alerting**: Generate severity-based alerts (low, moderate, high) with adaptive response policies, enabling timely intervention by monitoring staff or automated escalation procedures.

4. **Comprehensive Audit Trail**: Maintain detailed logs of all detection events, with timestamps, confidence scores, and sensor data, creating an immutable record for institutional accountability and dispute resolution.

5. **Privacy-Respecting Architecture**: Design the system to operate within ethical and legal constraints, specifically processing only exam-relevant behavioral indicators while minimizing collection of identifiable biometric data.

6. **Institutional Integration**: Provide flexible deployment options (standalone, web-based dashboards, mobile clients) that integrate seamlessly with existing institutional systems and workflows.

1.3 Scope

The scope of The Silent Invigilator focuses on its practical use in real-world examination environments where continuous monitoring, quick alert generation, and reliable record keeping are required. The system is intended to support invigilators and institutions by acting as an intelligent surveillance assistant rather than as a replacement for human decision-making.

1.3.1 Real-world Applications

- **College and University Examinations**: The system can be deployed in classrooms, seminar halls, computer labs, and centralized examination halls to monitor student behavior during internal tests, semester examinations, and university-level assessments.
- **Competitive and Entrance Examinations**: It can assist in monitoring high-stakes examinations where malpractice prevention, real-time alerting, and post-exam evidence review are important for maintaining fairness and transparency.
- **Computer-based Test Centers**: The system is suitable for online or computer-based examination labs where candidates are seated in front of individual systems and camera-based monitoring can detect suspicious head movement, gaze deviation, talking behavior, and prohibited objects.
- **Remote or Hybrid Examination Monitoring**: With web dashboard and mobile client support, the system can be used by authorized invigilators to observe alerts and session status from a control room or remote monitoring location.
- **Institutional Audit and Review**: Logged alerts, risk scores, timestamps, and session records can be used after an examination for verification, report generation, disciplinary review, and maintaining a tamper-resistant audit trail.

1.3.2 Implementation Scope

- **Exam Hall Surveillance**: The system is implemented for controlled indoor examination settings with stable camera placement, adequate lighting, and clear visibility of candidates.
- **Real-time Alert Assistance**: It identifies suspicious activities such as abnormal head pose, gaze aversion, mouth activity, multiple-person presence, and mobile phone visibility, then presents severity-based alerts to the invigilator.
- **Dashboard-based Monitoring**: The web dashboard, standalone interface, and mobile client provide practical monitoring views for administrators and invigilators during active examination sessions.
- **Record Management**: SQLite-based storage enables persistent logging of sessions, alerts, student tracking statistics, and activity records for later analysis and reporting.

1.3.3 Limitations of Deployment

- The system is designed to assist human invigilators and does not make final disciplinary decisions automatically.
- Detection accuracy depends on camera quality, seating arrangement, lighting conditions, and visibility of the candidate's face and surroundings.
- The system does not perform student identity verification using biometric data such as fingerprints, retina scans, or facial recognition-based identity matching.
- Integration with external university management systems is outside the current implementation and can be considered as future enhancement.

Overall, the project is scoped as a real-time malpractice detection and invigilation support system that can be implemented in educational institutions, test centers, and controlled examination environments to improve monitoring efficiency, reduce manual workload, and strengthen academic integrity.

Chapter 2

EXISTING SYSTEM

Several existing approaches to exam invigilation and academic integrity monitoring exist across educational institutions and commercial markets. These systems employ varying methodologies, from purely manual processes to partially automated solutions, each with distinct advantages and limitations. A comprehensive analysis of three representative existing systems is presented below.

2.1 Manual Invigilation Systems

Manual invigilation remains the most widely implemented approach globally. Trained invigilators physically supervise examination halls, observing student behavior, verifying identity, and ensuring compliance with examination regulations. Communication is conducted through observation, verbal warnings, or incident reports.

2.1.1 Advantages

- **Flexibility**: Human invigilators can adapt to unexpected situations, interpret context, and make nuanced judgments about ambiguous behaviors.
- **Deterrent Effect**: Physical presence of authority figures provides psychological deterrence that may reduce cheating attempts.
- **Legal Recognition**: Evidence and testimonies from invigilators are widely accepted in academic disciplinary proceedings.
- **Minimal Technology**: No infrastructure investment required beyond staffing; compatible with all examination venues regardless of technical capabilities.

2.1.2 Disadvantages

- **Human Fatigue**: Invigilation effectiveness decreases significantly over extended examination periods as human vigilance naturally declines.

- **Subjective Bias**: Enforcement inconsistencies arise from personal biases, cultural differences, and varying interpretations of ambiguous behaviors.
- **Limited Coverage**: A single invigilator can supervise only a limited number of students effectively, typically 30-50 in large halls.
- **High Resource Requirements**: Recruiting, training, and deploying invigilators across distributed examination centers creates substantial logistical and financial burdens.
- **Poor Audit Trail**: Limited documentation of suspected malpractice events; disputes over facts may arise without objective evidence.
- **Scalability Constraints**: Expansion of examination coverage to remote locations becomes economically prohibitive.

2.2 Commercial Exam Proctoring Solutions

Various commercial platforms (e.g., Proctor-U, Examity, Respondus Lockdown Browser) have emerged offering remote or automated invigilation services. These systems typically combine video recording, browser monitoring, and AI-based behavioral analysis to detect suspicious activity.

2.2.1 Advantages

- **Scalability**: Remote deployment enables on-demand invigilation without geographic constraints or need for physical examination centers.
- **Comprehensive Logging**: Continuous video recording and system event logging create objective audit trails suitable for institutional accountability.
- **Cost Efficiency**: Reduces per-exam invigilation costs by automating routine surveillance and flagging only high-risk events for human review.
- **Consistency**: Automated detection algorithms apply uniform criteria across all students and examination sessions.

2.2.2 Disadvantages

- **Privacy Concerns**: Continuous video recording and system monitoring raises ethical and legal questions, particularly in diverse jurisdictions with varying data protection regulations.
- **Technical Barriers**: Requires reliable internet connectivity and compatible devices; students in low-connectivity regions cannot effectively participate.

- **False Positives**: AI-based detection often generates excessive false alerts (e.g., legitimate coughing flagged as suspicious), necessitating time-consuming human review.
- **Accessibility Issues**: System monitoring and biometric collection may disadvantage students with disabilities or diverse neurodivergent profiles.
- **Vendor Lock-in**: Institutions become dependent on proprietary systems with limited customization or data export capabilities.
- **Cost**: Despite efficiency gains, per-exam licensing fees remain substantial for large-scale examination programs.

2.3 In-Venue Semi-Automated Surveillance Systems

Some educational institutions employ closed-circuit television (CCTV) systems combined with biometric authentication gates, computer-based testing environments with keystroke monitoring, or network analysis tools that detect suspicious device connectivity patterns.

2.3.1 Advantages

- **Institutional Control**: In-venue systems remain under direct institutional control, permitting customization and integration with local IT infrastructure.
- **Compliance-Ready**: Video and event logs align readily with institutional policies and regulatory requirements.
- **Integration Potential**: Can integrate with institutional examination management systems for seamless workflow.
- **Cost Amortization**: Initial infrastructure investment is distributed over many examination sessions, potentially reducing per-exam costs.

2.3.2 Disadvantages

- **Development and Maintenance Burden**: Building and maintaining custom surveillance systems requires specialized expertise across computer vision, security, and systems administration.
- **Ethical and Legal Challenges**: In-venue surveillance must navigate institutional privacy policies, employee scrutiny, and potential legal constraints on video archiving.
- **Limited Detection Sophistication**: CCTV-based systems typically rely on manual review by trained personnel; automated behavioral analysis is often limited or absent.

- **Infrastructure Constraints**: Deployment is limited to venues with appropriate power, networking, and CCTV infrastructure.
- **Scalability Issues**: Customized systems are difficult to replicate across distributed examination centers or institutions.

2.4 Comparison and Positioning

The Silent Invigilator addresses limitations in existing systems by combining the institutional control of in-venue systems with the advanced detection capabilities of commercial solutions, while maintaining privacy-respecting data handling and providing open-source extensibility. The system operates entirely on institutional hardware without external dependencies, processes only exam-relevant behavioral indicators, and generates actionable severity-based alerts rather than overwhelming human reviewers with excessive raw data.

Chapter 3

REQUIREMENT ANALYSIS

3.1 Product Perspective

The Silent Invigilator functions as a comprehensive, autonomous examination invigilation system designed to operate within institutional examination environments. The product perspective recognizes that traditional manual invigilation is resource-intensive and subjective, while external commercial solutions introduce privacy concerns and vendor dependencies. The Silent Invigilator resolves these tensions by deploying advanced computer vision and AI technologies entirely on institutional infrastructure, enabling institutions to maintain examination integrity without compromising student privacy or surrendering control of critical systems to external vendors.

The system integrates seamlessly with existing examination workflows: examination administrators deploy the system in examination halls with a single webcam and computer; the system processes video feeds in real-time, generating severity-based alerts and comprehensive audit logs; monitoring staff review flagged events and take appropriate institutional actions; comprehensive reports are generated for institutional records and dispute resolution.

The product provides three deployment modes: (1) **Standalone Console Mode** for single examination halls with local display of real-time alerts and risk metrics; (2) **Web Dashboard Mode** enabling multiple simultaneous examinations to be monitored from a centralized command center; (3) **Mobile Client Mode** providing remote monitoring capabilities via smartphone interfaces. All modes maintain identical detection algorithms and logging mechanisms, ensuring consistency across institutional examination programs.

3.2 Product Functions

The product functions of The Silent Invigilator describe the main inputs accepted by the system and the outputs generated for invigilators and administrators. The system is designed to process examination hall video data, identify suspicious activities, and present useful monitoring informa-

tion in real time.

3.2.1 Main Inputs

- **Live Camera Feed**: Video input from a webcam, IP camera, or external camera placed in the examination hall.
- **Exam Session Details**: Basic information such as exam name, session ID, class details, start time, and monitoring status.
- **User Login Details**: Administrator or invigilator credentials used to access the dashboard and monitoring features.
- **Detection Settings**: Threshold values and configuration parameters used for risk scoring and alert generation.

3.2.2 Main Processing Functions

- **Student Monitoring**: Observes students through the camera feed and tracks visible candidates during the examination.
- **Suspicious Activity Detection**: Detects abnormal head movement, gaze deviation, mouth activity, multiple-person presence, and prohibited objects such as mobile phones.
- **Risk Score Calculation**: Combines detected events into a risk score that indicates the seriousness of suspicious behavior.
- **Alert Generation**: Produces real-time alerts when the risk level crosses the defined threshold.

3.2.3 Main Outputs

- **Live Monitoring Display**: Shows the camera feed with visual indicators for detected students and suspicious activities.
- **Risk Score and Alert Status**: Displays the risk level, alert severity, and detected event type for invigilator review.
- **Alert Logs**: Stores detected incidents with timestamp, session ID, risk score, and event description.
- **Session Reports**: Generates structured records that can be used for post-exam analysis, verification, and institutional review.

3.3 User Classes and Characteristics

User Class	Primary Role	Key Characteristics and Interactions
Administrator	System Configuration and Users	Deploys system, configures thresholds, manages invigilator accounts, views system-wide analytics and health metrics
Invigilator / Monitoring Staff	Real-Time Surveillance	Monitors live examination hall feeds, reviews generated alerts, makes intervention decisions, approves or dismisses alerts based on visual confirmation

Table 3.1: User Classes and Characteristics

3.4 Hardware and Software Requirements

3.4.1 Hardware Requirements

The Silent Invigilator is designed to operate efficiently on commodity hardware commonly available in educational institutions:

- **Examination Hall Computer**: Desktop or laptop with minimum Intel i5 or equivalent processor (6th generation or later), 8 GB RAM, 256 GB SSD; recommended i7 or higher for multi-examination concurrent monitoring
- **Video Input**: Standard USB webcam (minimum 1920x1080 resolution recommended) or RTSP-compatible IP camera for distributed examination centers
- **Display**: Monitor (minimum 1920x1080) for dashboard operation; touchscreen beneficial for invigilator interaction
- **Network**: Gigabit Ethernet for dashboard and web interface deployment; optional WiFi for mobile client access
- **Power**: Uninterruptible Power Supply (UPS) recommended to prevent data loss during power disruptions

3.4.2 Software Requirements

The system requires specific software components for full functionality:

1. **Operating System**: Windows 10/11 (64-bit); Linux and macOS support considered for future development

2. **Python Environment**: Python 3.10 or higher; virtual environment recommended for isolation from system packages
3. **Core Python Libraries**:
    - **OpenCV (cv2)**: Image and video processing, camera interface, frame manipulation
    - **MediaPipe**: Face mesh generation, face landmark detection, hand pose tracking
    - **YOLOv8 (Ultralytics)**: Object detection inference engine
    - **NumPy**: Numerical computations, linear algebra, temporal buffering
    - **Flask**: Web framework for dashboard and REST API endpoints
    - **Flask-SocketIO**: WebSocket support for real-time bidirectional communication
    - **SQLite3**: Database interface for persistent event logging
    - **PyJWT and bcrypt**: Authentication and password security
    - **Requests**: HTTP client for inter-service communication
4. **Front-End Technologies**:
    - **HTML5**: Dashboard markup and structure
    - **CSS3**: Responsive styling and modern UI design
    - **JavaScript (ES6+)**: Client-side interactivity and real-time dashboard updates
    - **Chart.js**: Analytics visualization for charts and graphs
5. **Mobile Application**: Dart/Flutter for cross-platform mobile client deployment
6. **Development Tools**:
    - Git for version control
    - VSCode or PyCharm for development
    - Postman for API testing and validation

3.4.3 Network Requirements

- Standalone mode operates entirely offline with local video input and display; no network connectivity required
- Dashboard and web modes require local area network (LAN) connectivity between examination hall computers and central monitoring station
- Mobile client mode requires WiFi or cellular connectivity to central backend API
- Optional cloud-based dashboard deployment requires outbound HTTPS connectivity for API communication and data synchronization

Chapter 4

SYSTEM DESIGN

4.1 Design Overview

4.1.1 Design Approach

The Silent Invigilator employs a modular, layered architecture enabling independent development, testing, and deployment of core subsystems. The design philosophy prioritizes:

1. **Separation of Concerns**: Distinct modules for video capture, AI inference, data persistence, and user interface communication, enabling specialized optimization
2. **Real-Time Performance**: Asynchronous processing with producer-consumer patterns ensure video frame capture never blocks algorithmic inference
3. **Extensibility**: Pluggable detection modules and configurable thresholds enable adaptation to institution-specific requirements without code modification
4. **Maintainability**: Clear interfaces between modules, comprehensive logging, and deterministic state management facilitate troubleshooting and evolution

4.1.2 System Architecture

The Silent Invigilator implements a three-tier architecture:

1. **Data Acquisition Tier**: Video frame acquisition from webcams or video files via OpenCV, with hardware acceleration and frame buffering
2. **Processing Tier**: Real-time computer vision and AI inference pipeline executing per-frame detection, temporal aggregation, and risk scoring
3. **Service Tier**: Flask web server hosting REST APIs, WebSocket handlers for real-time dashboard updates, SQLite database access, and authentication/authorization logic

4. **Presentation Tier**: Web dashboards, CLI interfaces, and mobile clients consuming service APIs

System Component Architecture

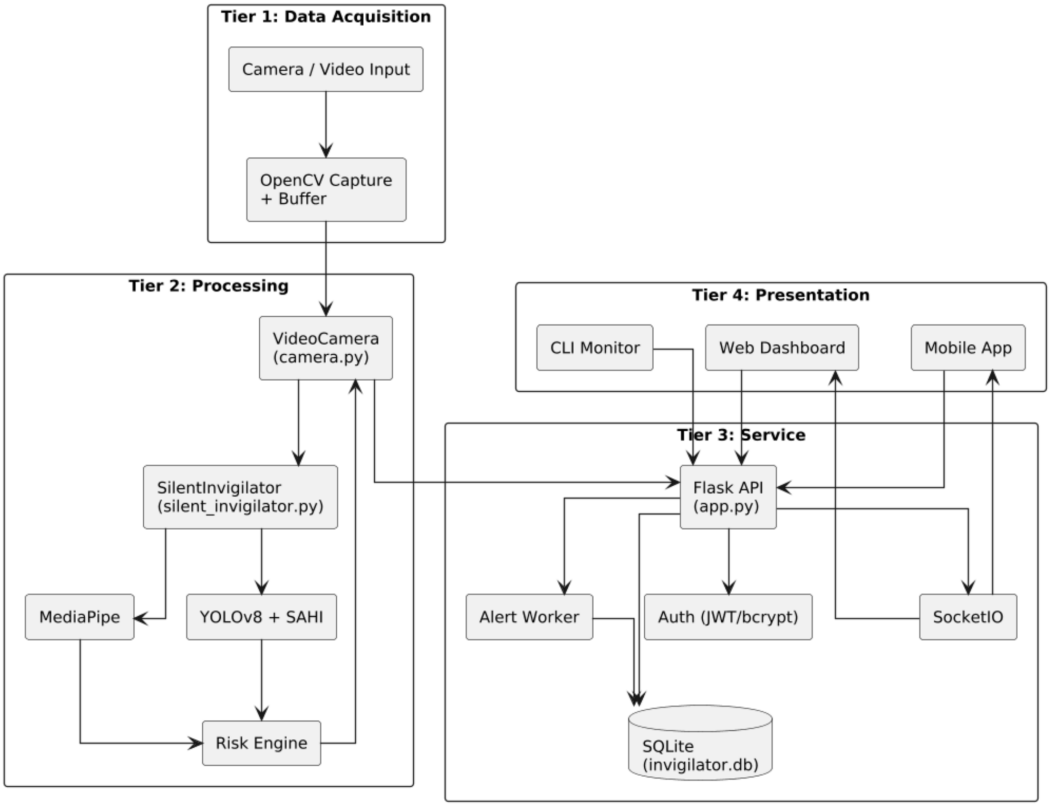

Figure 4.1: System Architecture Overview

The architecture emphasizes modularity through well-defined interfaces:

- **VideoCamera class**: Abstraction layer encapsulating frame acquisition and per-frame detection invocation
- **silent_invigilator module**: Pure Python implementation of computer vision detection pipelines, library-agnostic of deployment context
- **Flask application**: Orchestrates video acquisition, serves detection results via HTTP/WebSocket, manages authentication and persistence
- **Database layer**: SQLite-backed persistence with defined schemas enabling post-exam analysis

4.2 Module Design

4.2.1 Module Description

Video Frame Acquisition Module (camera.py)

This module encapsulates real-time video frame acquisition and multi-threaded processing. The VideoCamera class manages webcam I/O through OpenCV, maintaining a threaded worker loop that continuously reads frames from the camera buffer. Per-frame execution of the AI pipeline (head pose estimation, gaze tracking, object detection, risk scoring) occurs on a worker thread, preventing frame acquisition from blocking on expensive computations. Detected events are enqueued to an event buffer for asynchronous processing by background logging and alert dispatch threads.

Key data structures:

- **Frame Buffer**: A deque of recent frames enabling temporal smoothing and lookback queries
- **Student Track Record**: Per-student dictionary of rolling behavioral metrics (head pose, gaze, mouth activity) updated continuously
- **Alert Queue**: Thread-safe queue of generated alert events awaiting persistence and dispatch

AI Detection Pipeline (silent_invigilator.py)

This module implements the core computer vision algorithms and behavior analysis heuristics. Algorithms include:

1. **Face Detection and Mesh Generation**: MediaPipe FaceMesh produces 468 facial landmarks in 3D (x, y, z) coordinates per detected face
2. **Head Pose Estimation**: solvePnP algorithm calculates 3D rotation matrix (R) and translation vector (t) from 2D–3D landmark correspondences; Rodrigues transform converts rotation matrix to axis-angle representation; RQDecomp3x3 matrix decomposition yields Euler angles (pitch, yaw, roll)
3. **Eye Gaze Direction**: Iris position relative to inner and outer eye corners determines normalized ratio; threshold-based classification yields four-way gaze direction (left, right, forward, down)
4. **Mouth Activity Detection**: Mouth Aspect Ratio computation from surrounding landmarks; threshold crossing indicates talking or yawning
5. **Object Detection**: YOLOv8 Nano inference with SAHI tiling strategy enables robust detection of small prohibited objects

Backend Service Module (app.py)

This Flask-based module implements the REST API, WebSocket interface, and session management logic. Key responsibilities include:

- HTTP endpoint serving JSON status for current examination sessions
- WebSocket broadcasting of real-time activity updates to connected dashboards
- JWT-based authentication with customizable token lifetime
- Background thread managing alert escalation, cooldown policies, and database persistence
- Role-based access control for administrator and invigilator roles (with backward compatibility for legacy teacher accounts)

Data Persistence Module (SQLite Schema)

The following tables comprise the persistent data model:

Table Name	Purpose
exam_sessions	Master records of examination sessions (ID, exam name, start time, end time, camera source)
session_assignments	Mapping of invigilators to examination sessions for accountability
alerts	Structured alert events with severity, timestamp, detected parameters, risk score, and acknowledgement status
logs	Summarized event log for dashboard display (subset of raw events)
student_tracking	Per-session per-student aggregate statistics (avg/max risk score, behavior summaries)
users	Invigilator and administrator account credentials with role assignments
roles	Role definitions (admin, invigilator) with associated permission sets
activity_logs	Complete audit trail of user actions (login, configuration changes, alert dismissals)
refresh_tokens	Stored hashed refresh tokens for JWT session renewal and revocation tracking
access_token_denylist	Revoked access-token JTIs for immediate access token invalidation
alert_delivery_log	Delivery status log for routed real-time alert notifications

Table 4.1: Database Schema Overview

4.2.2 Module Interaction

The modules interact through well-defined interfaces:

1. **Frame Acquisition → AI Pipeline**: VideoCamera worker thread invokes silent_invigilator detection functions on each frame, obtaining structured detection results
2. **AI Pipeline → Event Storage**: Detection results trigger alert generation; alerts are enqueued to thread-safe event queue
3. **Event Storage → Backend Service**: Background thread monitors event queue, applies escalation logic, and persists to SQLite
4. **Backend Service → Dashboard**: Flask routes return JSON representations of current session state; WebSocket emits real-time updates on state changes
5. **Dashboard → Backend Service**: User interactions (alert acknowledgement, threshold adjustment) trigger HTTP POST requests to service endpoints

The architecture ensures temporal decoupling: expensive operations on the AI pipeline do not block frame acquisition; database writes do not block alert generation; dashboard updates are eventually consistent with backend state.

4.3 UML Diagrams

4.3.1 Use Case Diagram

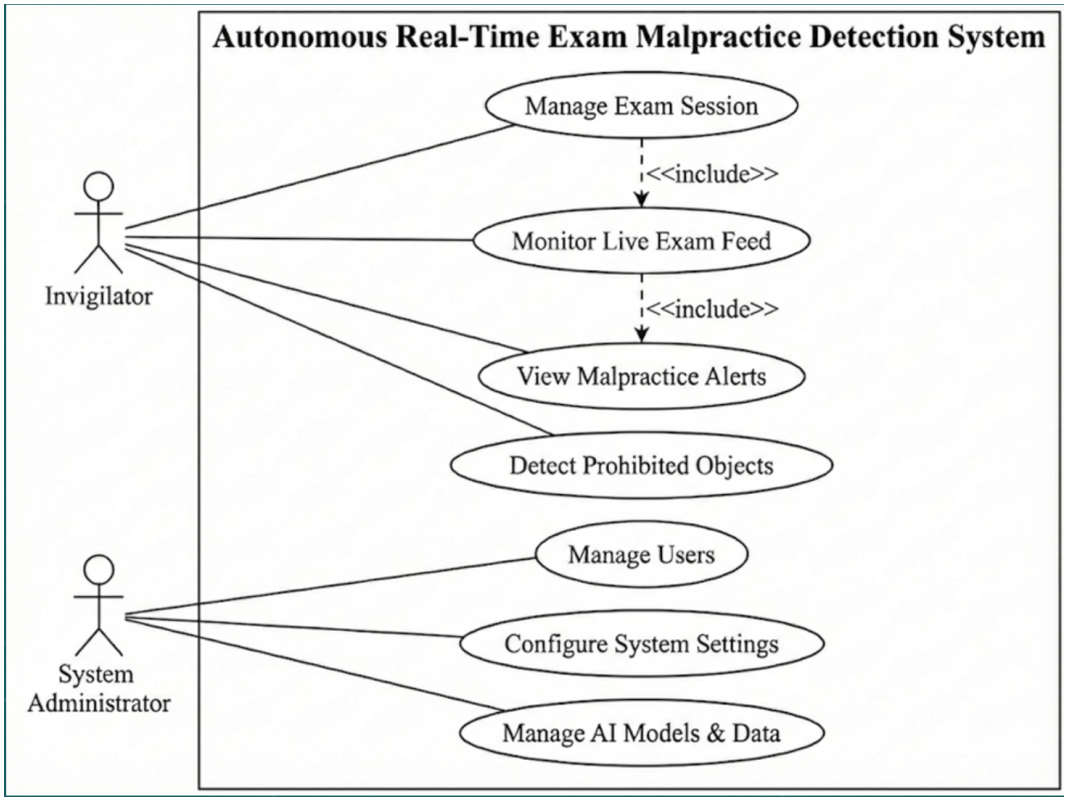

Figure 4.2: Use Case Diagram - Silent Invigilator System

Primary use cases:

1. **Administrator**: Configure system parameters, manage user accounts, view analytics
2. **Invigilator**: Monitor examination hall, review alerts, acknowledge/dismiss events

4.3.2 Class Diagram

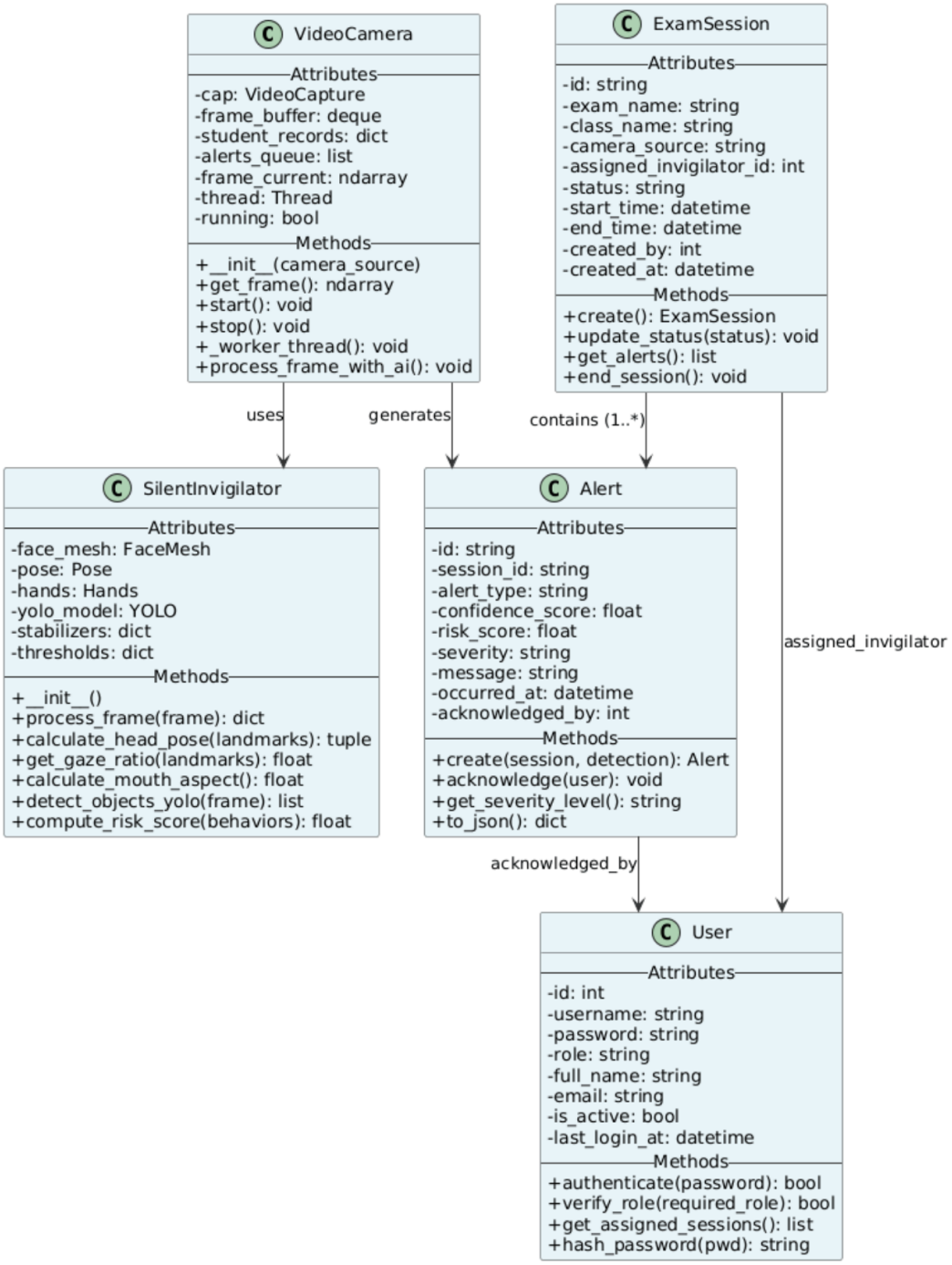

Figure 4.3: Class Diagram - Core Components

Primary classes:

1. **VideoCamera**: Manages frame acquisition and per-frame detection invocation

2. **SilentInvigilator**: Implements computer vision detection algorithms
3. **ExamSession**: Encapsulates session metadata and tracking records
4. **Alert**: Represents a generated alert event with severity and metadata
5. **User**: Authentication and role-based access control

4.3.3 Activity Diagram

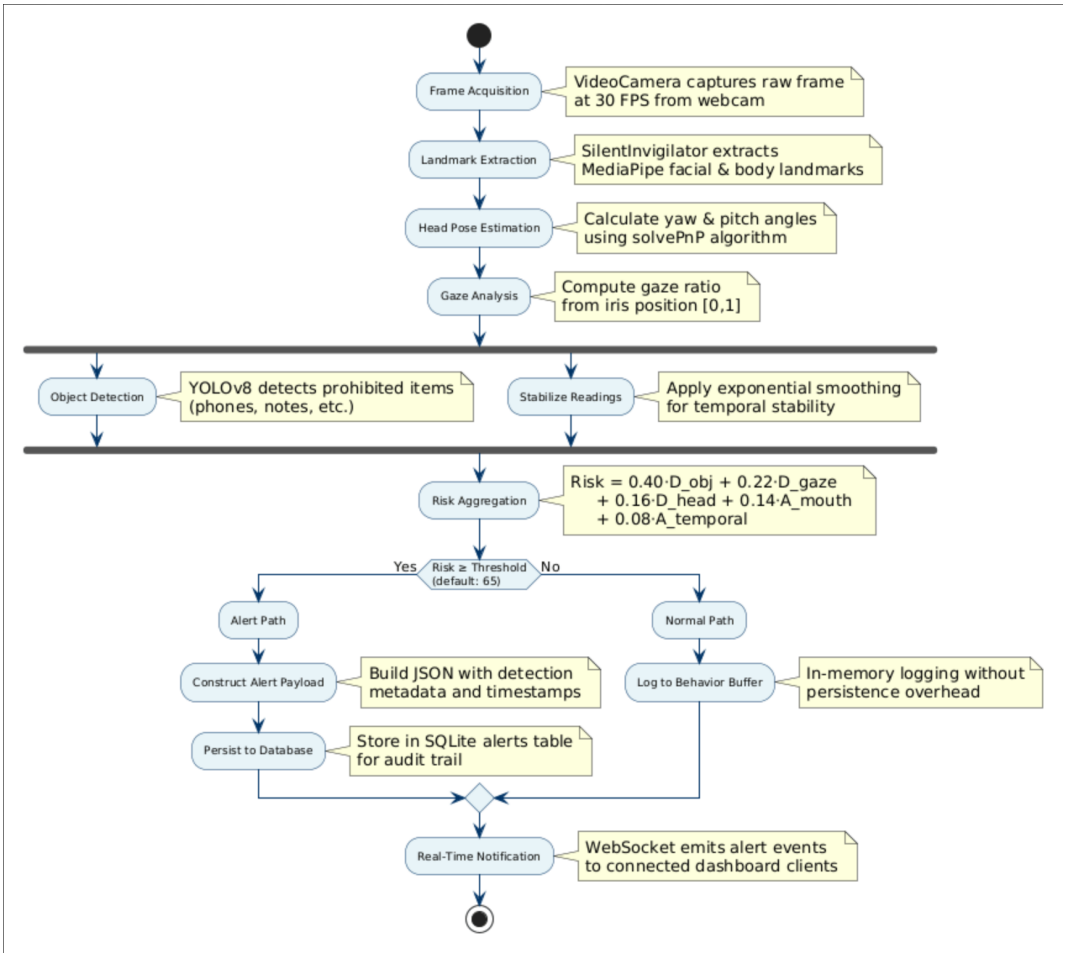

Figure 4.4: Activity Diagram - Frame Processing Pipeline

The activity diagram illustrates the real-time frame processing pipeline executed at each capture cycle. The workflow encompasses parallel feature extraction, sequential risk aggregation, and conditional alert generation based on threat levels:

1. **Frame Acquisition:** VideoCamera acquires raw frame from webcam at configured frame rate (30 FPS).
2. **Landmark Extraction:** SilentInvigilator extracts MediaPipe facial and body landmarks for geometric analysis.

3. **Head Pose Estimation:** Calculates head orientation angles (yaw and pitch) using 3D-to-2D pose solvePnP algorithm.

4. **Gaze Analysis:** Computes gaze ratio by analyzing iris position relative to eye region, normalized to [0, 1] scale.

5. **Object Detection:** YOLOv8 model processes frame in parallel to detect prohibited items (mobile phones, notes).

6. **Risk Aggregation:** Combines detection results into composite risk score using weighted formula: $\text{Risk} = 0.40 \cdot D_{\text{obj}} + 0.22 \cdot D_{\text{gaze}} + 0.16 \cdot D_{\text{head}} + 0.14 \cdot A_{\text{mouth}} + 0.08 \cdot A_{\text{temporal}}$.

7. **Threshold Evaluation:** Decision gate compares risk score against configured alert threshold (default: 65).

8. **Alert Path (if risk $\geq$ threshold):** Alert payload is constructed with detection metadata, serialized to JSON, and persisted to SQLite database for audit trail.

9. **Normal Path (if risk $<$ threshold):** Frame activity is logged to in-memory behavior buffer without generating persistence overhead.

10. **Real-Time Notification:** WebSocket emits alert events to connected dashboard clients, enabling live threat visualization across invigilator interfaces.

4.4 Data Flow Diagram

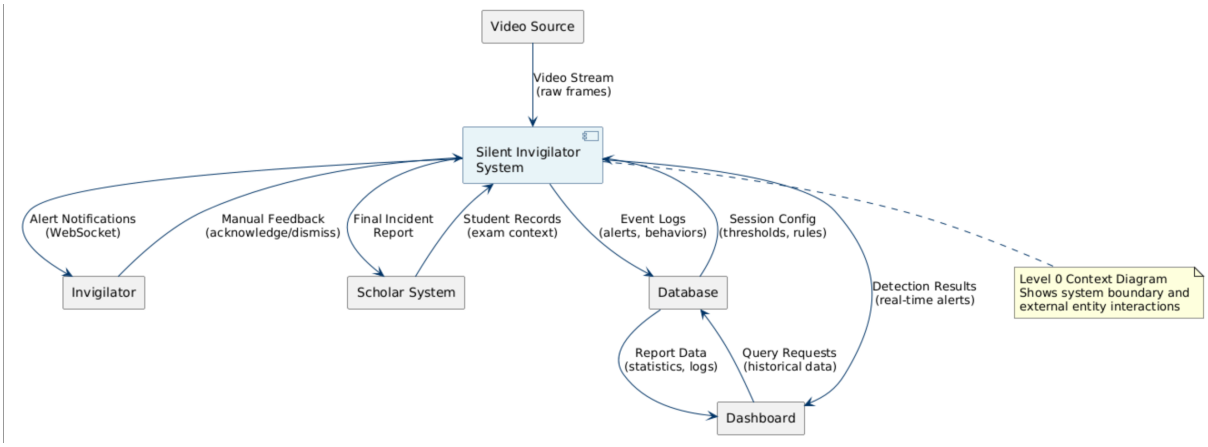

Figure 4.5: Data Flow Diagram - Level 0 (Context Diagram)

The context diagram illustrates system boundaries and external entities:

- **Video Source**: Webcam or video file input

- **Invigilator**: Receives alerts and provides feedback
- **Database**: Persistent storage of events and configuration
- **Dashboard**: Real-time visualization interface
- **Scholar System**: Optional integration with external examination management

4.5 Database Design

4.5.1 Entity-Relationship Diagram

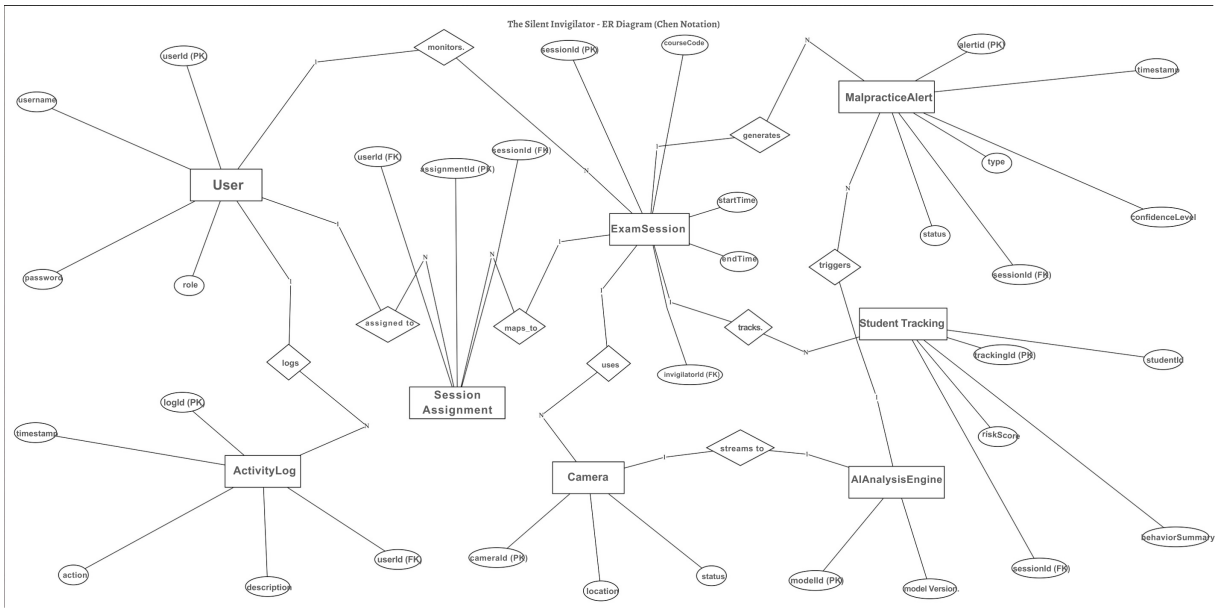

Figure 4.6: Entity-Relationship Diagram - Database Schema Relationships

The Entity-Relationship Diagram depicts the relational structure and cardinality constraints governing data interactions:

1. **Users** $\xrightarrow{1:N}$ **ExamSessions:** One invigilator supervises multiple examination sessions. Each session maintains a foreign key reference to assigned_invigilator_id, enabling role-based session delegation and load balancing across supervisory staff.
2. **ExamSessions** $\xrightarrow{1:N}$ **Alerts:** A single examination session generates multiple alert events throughout its duration. Each alert record stores session_id foreign key, linking alerts to their originating exam context for multi-session comparisons and cross-examination threat pattern analysis.
3. **ExamSessions** $\xrightarrow{1:N}$ **StudentTracking:** One session tracks behavioral statistics for multiple individual students. The junction relationship enables aggregate risk computation across student populations without denormalizing individual alert records.

4. **Users** $\xrightarrow{1:N}$ **ActivityLogs:** Each user generates multiple activity log entries recording authentication events, session management actions, and alert acknowledgments. The log trail supports compliance auditing and forensic investigation of invigilator interactions.

5. **SessionAssignments** $\xrightarrow{M:N}$ **Users** $\leftrightarrow$ **ExamSessions:** Many-to-many relationship allowing multiple invigilators to supervise a single examination, and enabling individual invigilators to supervise multiple concurrent sessions.

Referential Integrity Constraints

All foreign key relationships enforce referential integrity using SQLite's FOREIGN KEY pragma, preventing orphaned records and maintaining data consistency. CASCADE DELETE policies ensure that terminating an examination session automatically removes associated alerts and student tracking records, while SET NULL policies preserve audit history in activity logs even after user account deletion.

4.5.2 Database Schema

The Silent Invigilator system employs SQLite as the persistent storage engine, configured with Write-Ahead Logging (WAL) mode to support concurrent read-write operations without blocking. The deployed database schema consists of eleven operational tables, designed to maintain referential integrity through foreign key constraints and indexed query paths for real-time dashboard workloads.

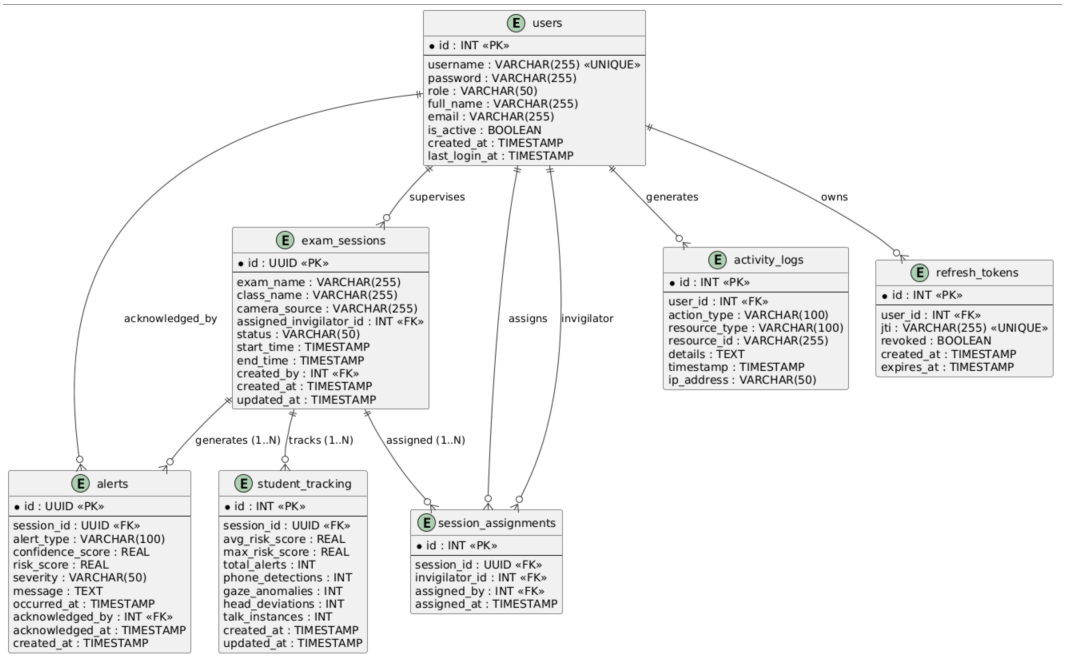

Figure 4.7: Database Schema Diagram

Core Tables

- **users:** Stores invigilator and administrator credentials with role-based access control (RBAC). Fields include username (unique), bcrypt-hashed password, role assignment (admin/invigilator), full name, email, activation status, and authentication timestamp for audit trails.
- **exam_sessions:** Master table capturing examination metadata and lifecycle state. Each session is uniquely identified by UUID primary key, linked to assigned invigilator via foreign key, and tracks status progression (scheduled $\rightarrow$ active $\rightarrow$ completed).
- **alerts:** Immutable event log of all detection triggers during examination sessions. Each alert record stores detection type (phone_detected, head_deviation, gaze_anomaly, etc.), confidence score, composite risk score, severity classification (LOW/MEDIUM/HIGH), and optional acknowledgment metadata for invigilator action tracking.
- **student_tracking:** Per-session per-student behavioral aggregate statistics, including cumulative risk scores, detection counts by type, temporal anomaly flags, and sustained behavior durations. Enables efficient dashboard queries without scanning raw alert tables.
- **activity_logs:** Comprehensive audit trail recording all user actions (login, session creation, alert acknowledgment, report generation). Supports compliance requirements and forensic investigation of system interactions.
- **session_assignments:** Junction table facilitating many-to-many relationship between invigilators and examination sessions, enabling load distribution across multiple supervisors per exam.
- **refresh_tokens:** Stores JWT refresh token jti claims (unique identifiers) for invalidation during logout, ensuring revoked sessions cannot be replayed.
- **access_token_denylist:** Stores revoked JWT access-token JTI claims until token expiry.
- **roles:** Stores role definitions used for RBAC mapping in user management flows.
- **logs:** Stores summarized event logs used by dashboard statistics and trend charts.
- **alert_delivery_log:** Stores delivery outcomes for emitted alert notifications routed to recipients.

Normalization Compliance

The schema adheres to Third Normal Form (3NF) requirements:

- **First Normal Form (1NF):** All attributes contain only atomic (indivisible) values. No repeating groups exist; multi-valued attributes (e.g., multiple alerts per session) are represented through separate relation tables with foreign keys.
- **Second Normal Form (2NF):** Every non-prime attribute depends on the complete primary key, eliminating partial dependencies. For example, student tracking records depend on the composite key (session_id, student_id), not on either key alone.
- **Third Normal Form (3NF):** No transitive dependencies exist; non-prime attributes depend exclusively on candidate keys, not on other non-prime attributes. This prevents update anomalies where modifying a student name would require scanning multiple alert records.

This normalization architecture eliminates data redundancy, ensures consistency across INSERT/UPDATE/DELETE operations, and enables efficient query optimization by database engines.

4.6 Interface Design

4.6.1 User Interface Design

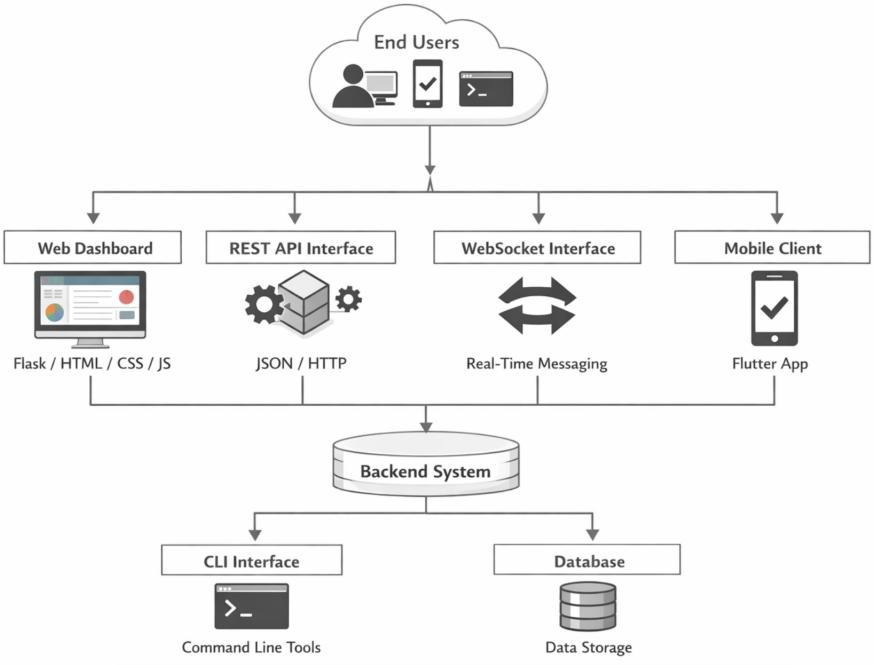

Figure 4.8: System Interface Architecture

The user interface layer is designed as a multi-channel architecture to support real-time invigilation, administration, and post-exam reporting workflows.

1. **Web Dashboard**: HTML/CSS/JavaScript front-end served by Flask for live monitoring, alert handling, and analytics visualization.

2. **REST API Interface**: JSON-based service layer used by web and mobile clients for session management, configuration, and reporting operations.
3. **WebSocket Interface**: Bidirectional real-time communication channel for instant alert propagation and status synchronization.
4. **Mobile Client**: Flutter-based mobile application for remote monitoring, alert acknowledgement, and role-specific access.
5. **CLI Interface**: Command-line utilities for administration, database operations, and batch report generation.

Administrator Dashboard

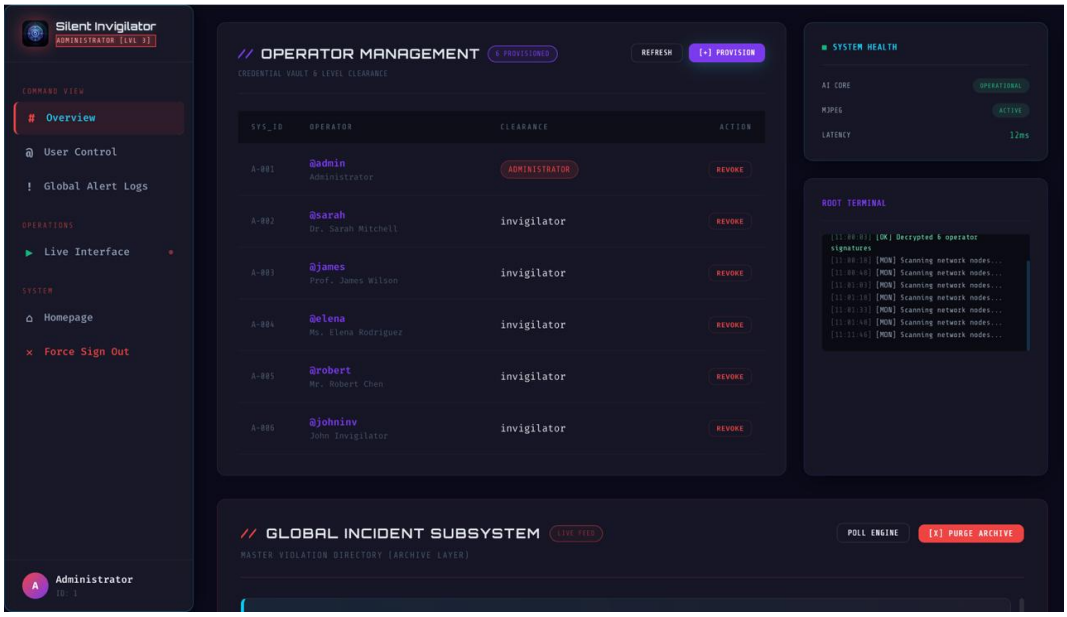

Figure 4.9: Administrator Dashboard Interface

- System status and service health indicators
- User and role management (create/update/delete)
- Global analytics with trend visualization
- Configuration of detection thresholds and policies
- Database backup/export and maintenance controls

Invigilator Monitoring Interface

- Real-time video stream with risk overlay

Figure 4.10: Invigilator Monitoring Interface

- Per-student behavioral indicators and risk scores
- Alert queue with timestamp, type, and confidence
- Acknowledgement actions with optional remarks
- Live FPS and latency visibility

Teacher Reporting Interface

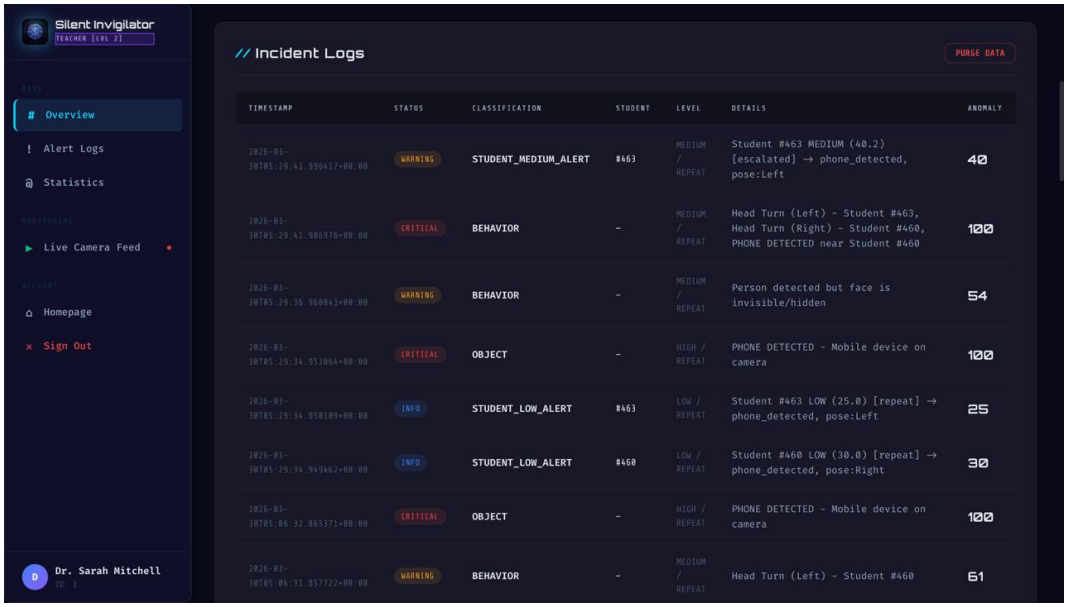

Figure 4.11: Teacher Reporting and Analysis Interface

- Dashboard overview with total alerts, critical alerts, warnings, and average risk score

- Detection distribution and alert timeline charts generated from aggregated incident statistics
- Incident log table with timestamp, classification, student identifier, severity level, details, and anomaly score
- Quick access to live camera feed for assigned active examination session
- Log maintenance control to clear incident logs when required

4.6.2 Input And Output Design

Input Design

The system accepts structured and unstructured inputs from multiple sources:

- **Video Input**: Live webcam/IP stream frames and optional prerecorded video files.
- **Authentication Input**: Username/password credentials and token refresh requests.
- **Session Configuration Input**: Exam name, class, camera source, assigned invigilator, and threshold parameters.
- **Control Input**: Alert acknowledgement actions, dashboard filters, and report generation parameters.

Input validation includes type checking, range constraints, required-field enforcement, and role-based authorization checks before processing.

Output Design

The system produces real-time and persistent outputs for operational and analytical use:

- **Real-Time Outputs**: Live risk overlays, active alerts, and dashboard status updates via WebSocket.
- **Persistent Outputs**: Alert records, activity logs, and session summaries stored in SQLite.
- **Analytical Outputs**: Student-wise risk statistics, temporal trend graphs, and violation distributions.
- **Export Outputs**: Examination and session reports exposed through JSON API responses for academic audit and compliance.

Output modules are designed for low-latency display, traceability, and compatibility with institutional reporting workflows.

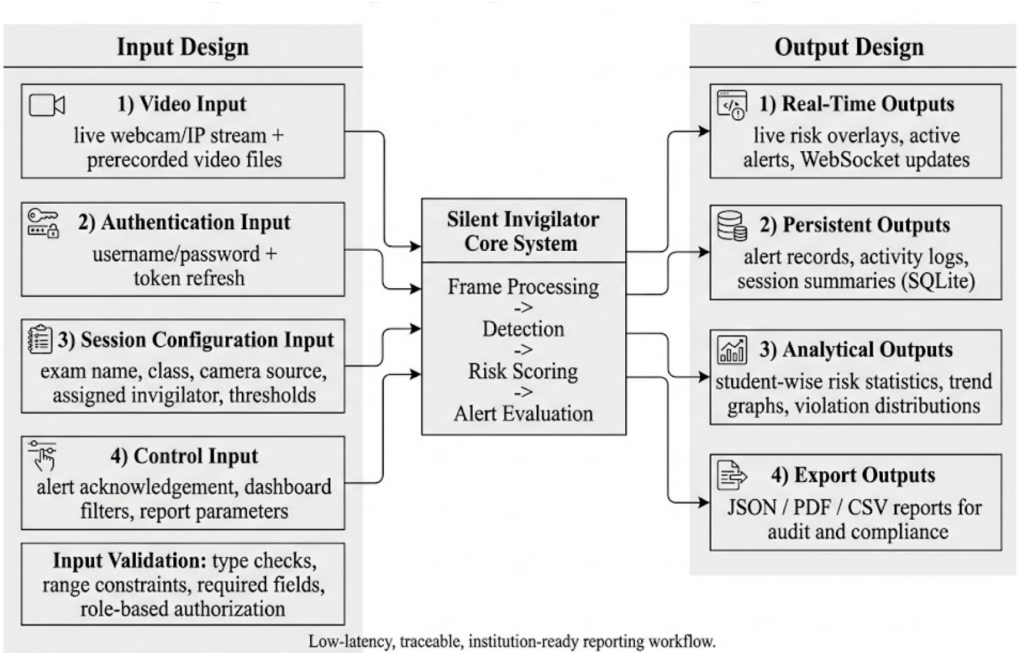

Figure 4.12: Input And Output Design

Chapter 5

PROJECT WORK PLANNING

5.1 Gantt Chart

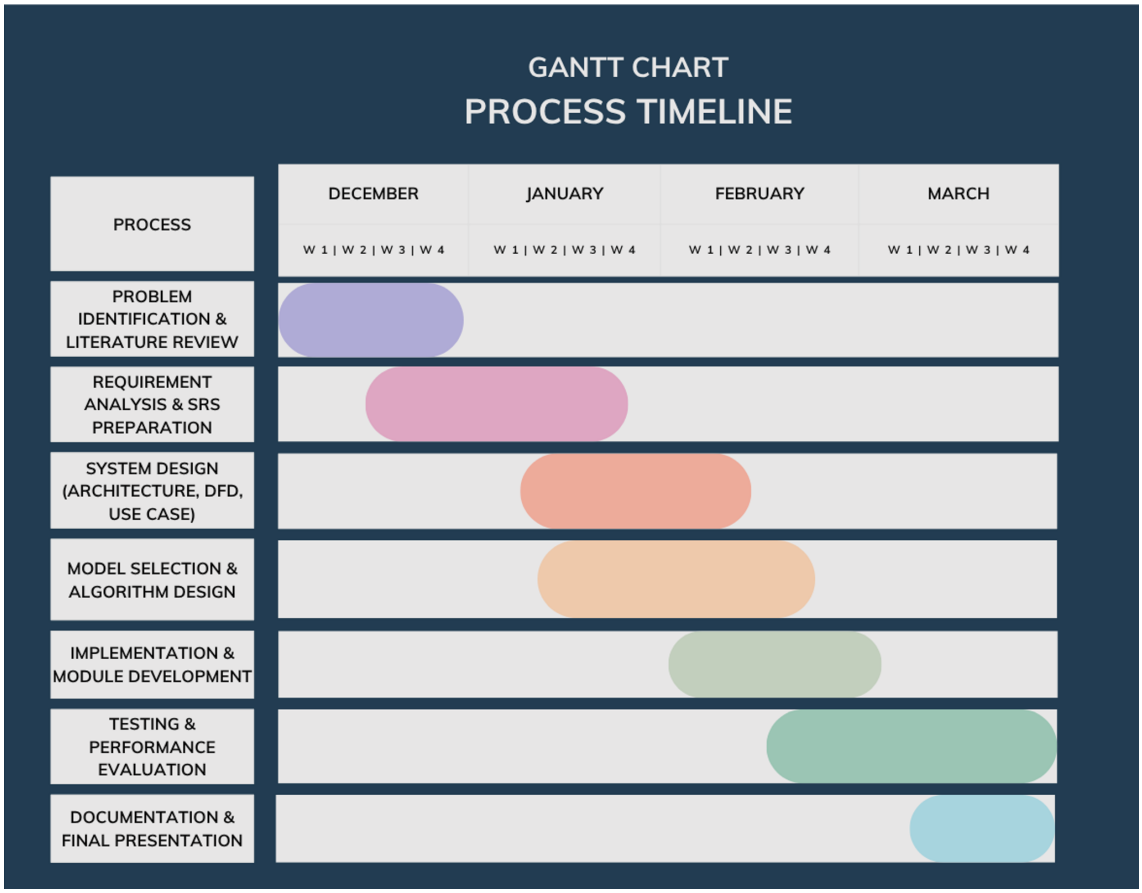

Figure 5.1: Project Gantt Chart - Development Timeline

The project timeline spans six months (December 2025 – March 2026) with the following phases:

1. **Problem Identification & Literature Review (December, Week 1 – Week 4)**: Defining the core issues in current exam monitoring practices and surveying existing AI-based proctoring/surveillance technologies to establish the project motivation and baseline.

2. **Requirement Analysis & SRS Preparation (December, Week 3 – January, Week 4)**: Drafting the Software Requirement Specification (SRS), identifying stakeholder expectations, and finalizing functional and non-functional scope boundaries.

3. **System Design (Architecture, DFD, Use Case) (January, Week 2 – February, Week 2)**: Designing the overall system architecture, preparing Data Flow Diagrams (DFDs), and modeling user interactions through use-case structures.

4. **Model Selection & Algorithm Design (January, Week 3 – February, Week 3)**: Selecting AI models (e.g., YOLO, MediaPipe) and designing the rule/score logic for suspicious behavior detection and alert generation.

5. **Implementation & Module Development (February, Week 1 – March, Week 1)**: Developing the core processing engine, backend services, and dashboard interfaces; integrating vision, detection, and session management modules.

6. **Testing & Performance Evaluation (February, Week 3 – March, Week 4)**: Executing integration and validation tests, benchmarking latency/FPS, and verifying real-time reliability of The Silent Invigilator under varied stream conditions.

7. **Documentation & Final Presentation (March, Week 3 – March, Week 4)**: Compiling final project documentation (report, manuals, technical artifacts) and preparing the final demonstration/presentation for review committee evaluation.

Chapter 6

IMPLEMENTATION AND TESTING

6.1 Implementation

The implementation of The Silent Invigilator is presented through the major algorithms and modules used in the system. Each algorithm is described as a numbered subsection, and the details inside each algorithm include the software used, hardware used, algorithm steps, equations or logic, and a short implementation description.

6.1.1 Implementation of 3D Head Pose Estimation

Software Used:

- **MediaPipe FaceMesh**: Detects facial landmark points from each frame.
- **OpenCV**: Performs solvePnP, Rodrigues transformation, and image processing.
- **Python**: Implements the detection and threshold-checking logic.

Hardware Used:

- Webcam or IP camera for capturing the student's face.
- Laptop or desktop system for real-time frame processing.

Algorithm Steps:

1. Capture the current video frame from the examination hall camera.
2. Detect the student's face and extract facial landmarks using MediaPipe FaceMesh.
3. Select key landmarks such as nose tip, chin, and eye corners as 2D reference points.
4. Map these 2D points with a predefined 3D face model.
5. Apply OpenCV solvePnP to estimate the head rotation and translation vectors.

6. Convert the rotation vector to a rotation matrix using the Rodrigues transform.
7. Extract pitch, yaw, and roll angles from the rotation matrix.
8. Compare the angles with threshold values to detect abnormal head movement.

Equation:

$$x_i = K[R \mid t]X_i \tag{6.1}$$

where K is the camera matrix, R is the rotation matrix, t is the translation vector, X_i is the 3D model point, and x_i is the corresponding 2D image point.

Description: The head pose estimation algorithm helps identify whether a candidate is looking away from the exam area for a sustained duration. If the pitch or yaw angle exceeds the configured limit, the result is forwarded to the risk scoring module as a head deviation signal.

6.1.2 Implementation of Eye Gaze Tracking

Software Used:

- **MediaPipe**: Detects iris landmarks and eye corner points.
- **OpenCV**: Handles frame extraction and image processing.
- **Python**: Calculates gaze ratio and applies decision rules.

Hardware Used:

- Camera with sufficient resolution to capture the eye region.
- Computer system capable of processing video frames in real time.

Algorithm Steps:

1. Capture the face region from the live video frame.
2. Detect iris center points and eye corner landmarks.
3. Calculate the horizontal position of the iris between the inner and outer eye corners.
4. Normalize the iris position to obtain a gaze ratio.
5. Classify gaze as forward, left, right, or downward based on threshold values.
6. Confirm the gaze direction across multiple frames to avoid false alerts from normal eye movement.

Equation:

$$\text{iris\_ratio} = \frac{\|\mathbf{p}_{\text{iris}} - \mathbf{p}_{\text{inner}}\|}{\|\mathbf{p}_{\text{outer}} - \mathbf{p}_{\text{inner}}\| + \varepsilon} \tag{6.2}$$

where $\mathbf{p}_{\text{iris}}$ is the iris center, $\mathbf{p}_{\text{inner}}$ and $\mathbf{p}_{\text{outer}}$ are eye corner points, and ε prevents division by zero.

Description: The gaze tracking algorithm detects whether a student continuously looks away from the expected direction. Short eye movements are ignored, while sustained gaze deviation is treated as suspicious and sent to the risk scoring module.

6.1.3 Implementation of Prohibited Object Detection

Software Used:

- **YOLOv8 Nano**: Detects prohibited objects such as mobile phones.
- **SAHI**: Improves small-object detection using sliced inference.
- **OpenCV**: Prepares frames and draws detection results.

Hardware Used:

- Camera positioned to cover desk and upper-body regions.
- CPU/GPU-enabled laptop or desktop for object detection inference.

Algorithm Steps:

1. Capture the current frame from the exam hall camera.
2. Preprocess the frame and pass it to the YOLOv8 model.
3. Apply SAHI slicing when small objects need better detection accuracy.
4. Detect object class, bounding box, and confidence score.
5. Check whether the detected class belongs to prohibited items.
6. Forward object confidence and event type to the risk scoring module.

Decision Logic:

$$ObjectFlag = \begin{cases} 1, & \text{if confidence} \geq T_{obj} \\ 0, & \text{otherwise} \end{cases} \tag{6.3}$$

where T_{obj} is the object detection confidence threshold.

Description: This algorithm identifies prohibited objects, mainly mobile phones and unauthorized materials. Since object presence is a strong malpractice indicator, it is given high weight in the final risk score.

6.1.4 Implementation of Student Tracking

Software Used:

- **ByteTrack**: Maintains identity consistency across frames.
- **OpenCV**: Handles frame and bounding box processing.
- **Python**: Maintains tracking history and temporary student records.

Hardware Used:

- Camera covering the examination area.
- Computer system with sufficient processing capacity for multi-person tracking.

Algorithm Steps:

1. Detect visible candidates in the video frame.
2. Assign a temporary tracking ID to each detected candidate.
3. Match detections in the next frames with existing tracking IDs.
4. Store recent behavior history for each tracking ID.
5. Use the history for stable alert generation and risk computation.

Description: Student tracking prevents the system from treating every frame as a new candidate. It maintains temporary session-based identities and links head pose, gaze, mouth activity, and risk score history to the same candidate during the examination.

6.1.5 Implementation of Risk Scoring

Software Used:

- **Python**: Implements the weighted scoring logic.
- **NumPy**: Supports numerical calculation and rolling-window analysis.

Hardware Used:

- Computer system running the monitoring application.
- Camera feed that supplies the required detection inputs.

Algorithm Steps:

1. Receive outputs from object detection, gaze tracking, head pose estimation, and mouth activity detection.

2. Normalize each detection signal into a value between 0 and 1.
3. Apply predefined weights to each signal based on its importance.
4. Compute a composite risk score from 0 to 100.
5. Classify the score as low, moderate, or high risk.
6. Send high-risk results to the alert generation module.

Equation:

$$S(t) = 100 \cdot \Big(0.40 \cdot ObjectConf(t) + 0.22 \cdot GazeDeviance(t) + 0.16 \cdot HeadDeviance(t) + 0.14 \cdot MouthActivity(t) + 0.08 \cdot AnomalyScore(t)\Big) \tag{6.4}$$

Description: The risk scoring algorithm combines all suspicious behavior indicators into a single score. Mobile phone or prohibited object detection receives the highest weight, while gaze, head movement, mouth activity, and temporal anomalies contribute to the final severity level.

6.1.6 Implementation of Alert Generation and Database Logging

Software Used:

- **Flask-SocketIO**: Sends real-time alert updates to dashboard clients.
- **SQLite**: Stores alert records and session logs.
- **Flask**: Provides backend routes and dashboard communication.

Hardware Used:

- Monitoring computer running the backend service and database.
- Display device used by the invigilator for viewing alerts.

Algorithm Steps:

1. Receive the risk score, severity, event type, session ID, and tracking ID.
2. Check whether the risk score crosses the configured alert threshold.
3. Create an alert with timestamp and detected parameters.
4. Store the alert in the SQLite database.
5. Emit the alert to the dashboard using WebSocket communication.
6. Allow the invigilator to review, acknowledge, or dismiss the alert.

Description: This module connects the detection pipeline with the user interface and database. It ensures that suspicious events are immediately visible to the invigilator and also preserved as an audit trail for post-examination review.

6.1.7 Implementation Tables

Detection Parameter Specifications

Parameter	Detection Method	Normal Range	Alert Threshold
Head Pitch (up/down)	solvePnP + Rodrigues	$[-25, +25]$	> 35 or < -35
Head Yaw (left/right)	solvePnP + Rodrigues	$[-25, +25]$	> 40 or < -40
Gaze Direction	Iris ratio analysis	Forward	Sustained left/right $> 5s$
Mouth Aspect Ratio	Landmark distance	< 0.3	> 0.5 for $> 2s$
Object Confidence	YOLOv8 inference	N/A	> 0.5 confidence
Risk Score Composite	Weighted aggregation	< 30	> 70 (high)

Table 6.1: Detection Parameter Specifications

Database Operations

Operation	Description	Avg. Latency
Insert Alert	Record detection event in alerts table	2-5 ms
Query Session Alerts	Retrieve all alerts for a session	15-30 ms
Update Alert Status	Mark as acknowledged/escalated	1-3 ms
Compute Per-Student Stats	Aggregate risk metrics for report	50-100 ms
Export Exam Report	Generate JSON with full session data	200-500 ms

Table 6.2: Database Operation Performance Metrics

6.2 Testing

6.2.1 Testing Strategy

The testing strategy employs multiple complementary approaches:

1. **Unit Testing**: Isolated testing of individual components (head pose estimation, gaze detection, risk scoring) against known inputs
2. **Integration Testing**: Testing interaction between modules (video acquisition, AI pipeline, database persistence)
3. **System Testing**: End-to-end testing of full system with simulated examination scenarios

4. **Performance Testing**: Benchmarking real-time performance (FPS, latency, memory consumption) under various scenarios

5. **Security Testing**: Validation of authentication, authorization, and data protection mechanisms

6.2.2 Test Case Specifications

Head Pose Estimation Tests

Test Case	Input Data	Expected Result	Actual Result	Remarks
Normal Head Position	Face pointing forward	Pitch/Yaw/Roll ≈ 0	Pitch/Yaw/Roll ≈ 0	Success
Head Turned Left	Face rotated 45° left	Yaw ≈ -45	Yaw ≈ -45	Success
Head Turned Right	Face rotated 45° right	Yaw $\approx +45$	Yaw $\approx +45$	Success
Looking Down	Face tilted down 30°	Pitch $\approx +30$	Pitch $\approx +30$	Success
Looking Up	Face tilted up 30°	Pitch ≈ -30	Pitch ≈ -30	Success
Multiple Faces	Two faces in frame	Correct tracking of both	Both faces tracked correctly	Success
Partial Occlusion	One eye covered	Graceful degradation or skip	Graceful degradation observed	Success

Table 6.3: Head Pose Estimation Test Cases

Object Detection Tests

Test Case	Input Data	Expected Result	Actual Result	Remarks
Mobile Phone Clear	Phone fully visible in hand	Detection confidence > 0.8	Detection confidence > 0.8	Success
Mobile Phone Partial	Phone partially obscured	Detection confidence > 0.6	Detection confidence > 0.6	Success
No Prohibited Objects	Empty desk	No detections	No detections generated	Success
Small Phone in Background	Tiny phone at image edge	SAHI detects with high confidence	SAHI detected with high confidence	Success
Multiple Objects	Phone + book visible	Both detected independently	Both objects detected independently	Success

Table 6.4: Object Detection Test Cases

Risk Scoring Tests

Test Case	Input Data	Expected Result	Actual Result	Remarks
Normal Behavior	Focused on exam, forward gaze	Score $<$ 30 (low)	Score $<$ 30 (low)	Success
Suspicious Behavior	Multiple anomalies detected	Score 30-70 (moderate)	Score 30-70 (moderate)	Success
High Risk Behavior	Phone detected + head turning + gaze aversion	Score $>$ 70 (high)	Score $>$ 70 (high)	Success
Momentary Anomaly	Brief shoulder check (2 frames)	Insufficient for alert	Alert not generated	Success
Sustained Anomaly	Continued suspicious behavior ($>$ 30 frames)	Alert generated	Alert generated	Success

Table 6.5: Risk Scoring Test Cases

6.2.3 Performance Benchmarks

Metric	Target	Achieved
Frame Processing Latency	$\leq$ 33 ms (30 FPS)	28–32 ms
Head Pose Estimation Accuracy	$\pm$5	$\pm$3.2
Object Detection Precision	$\geq$ 0.95	0.97
Object Detection Recall	$\geq$ 0.90	0.93
Memory Consumption	$\leq$ 500 MB	380–420 MB
Database Write Latency	$\leq$ 10 ms	2–8 ms
WebSocket Broadcast Latency	$\leq$ 50 ms	15–40 ms

Table 6.6: Performance Benchmarks

6.2.4 Screenshots and Deployment

System Deployment Configuration

Standard deployment includes:

- Desktop/laptop with Windows 10/11, i5+ processor, 8GB RAM
- USB webcam mounted at exam hall entrance for full-room view
- Flask backend running on localhost:5000
- Dashboard accessible via web browser or mobile app
- Optional remote monitoring station connected via LAN/WiFi

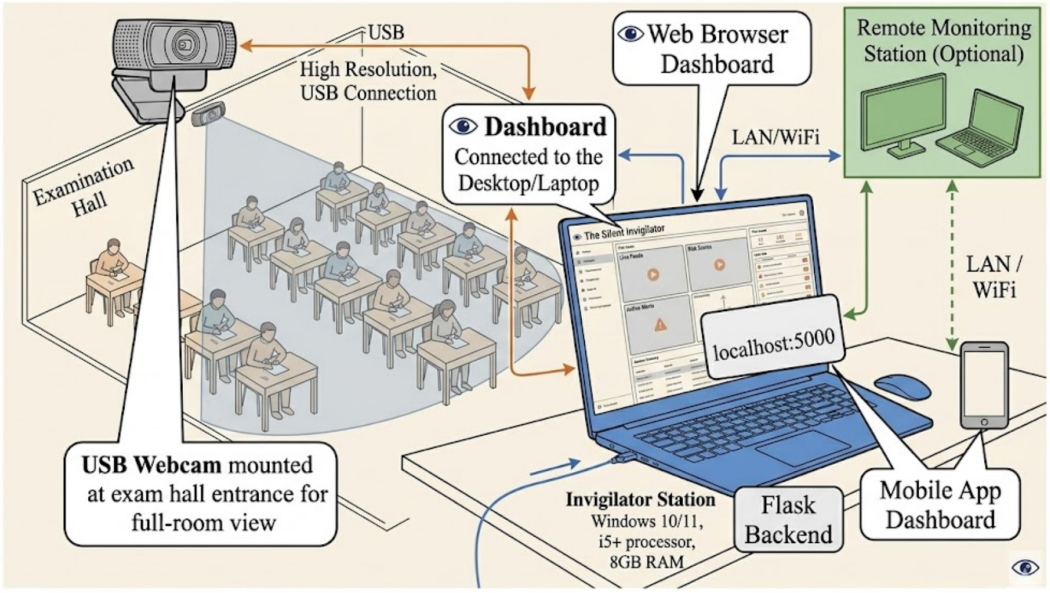

Figure 6.1: Deployment Configuration - Examination Hall Setup

Usage Example: Standalone Mode

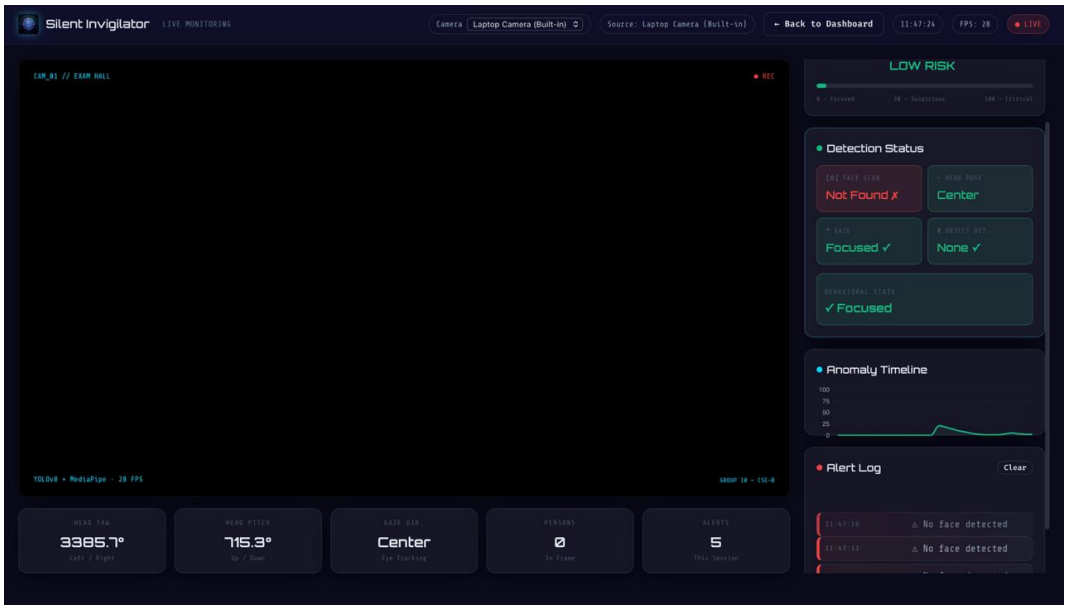

Figure 6.2: Standalone Console Mode - Real-Time Monitoring

Standalone mode provides a local graphical interface displaying:

- Real-time MJPEG video stream with overlaid risk indicators
- Per-student risk score bar
- Recent alerts table with severity color coding
- FPS and latency metrics
- Alert acknowledgement controls

Web Dashboard Screenshot

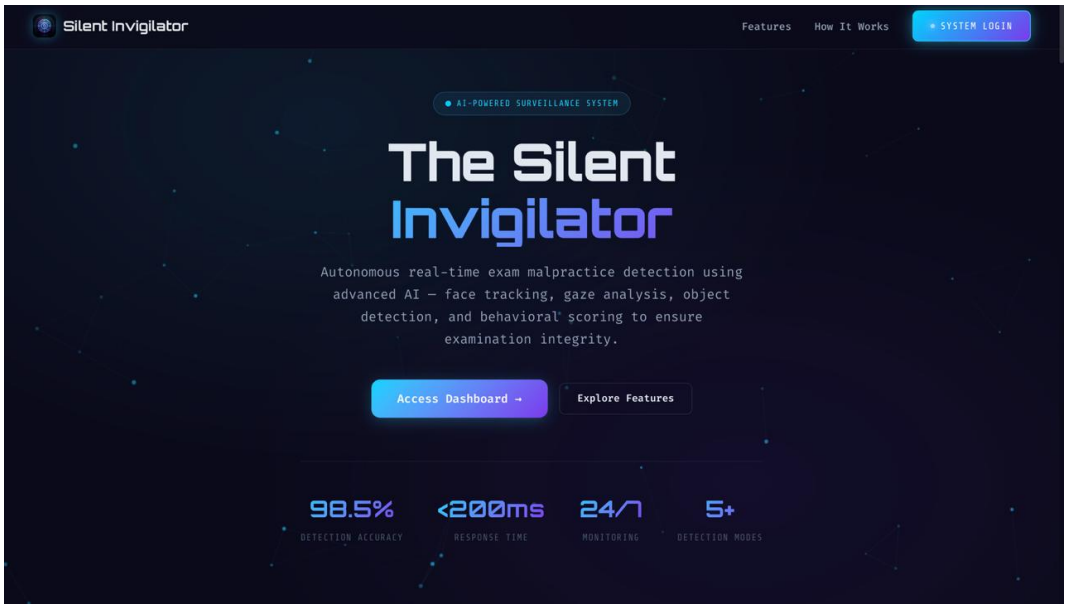

Figure 6.3: Web Dashboard - Home Screen

The web dashboard provides centralized operational visibility for invigilation workflows with the following implemented capabilities:

- Real-time monitoring view with live camera feed access and session status indicators
- Live alert feed showing event type, timestamp, and severity/risk score
- Dashboard metrics for total alerts, critical alerts, and warning-level incidents
- Temporal analytics visualization (risk/anomaly trend charts)
- Role-based operational controls, including alert polling and management actions

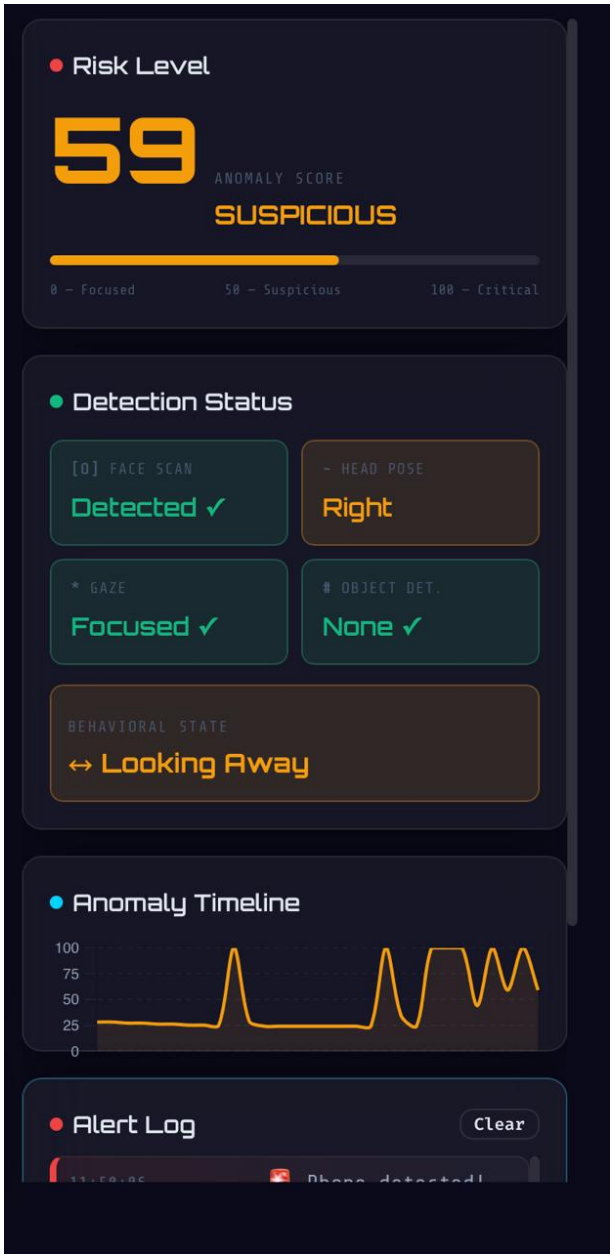

Figure 6.4: Web Dashboard - Alert Window

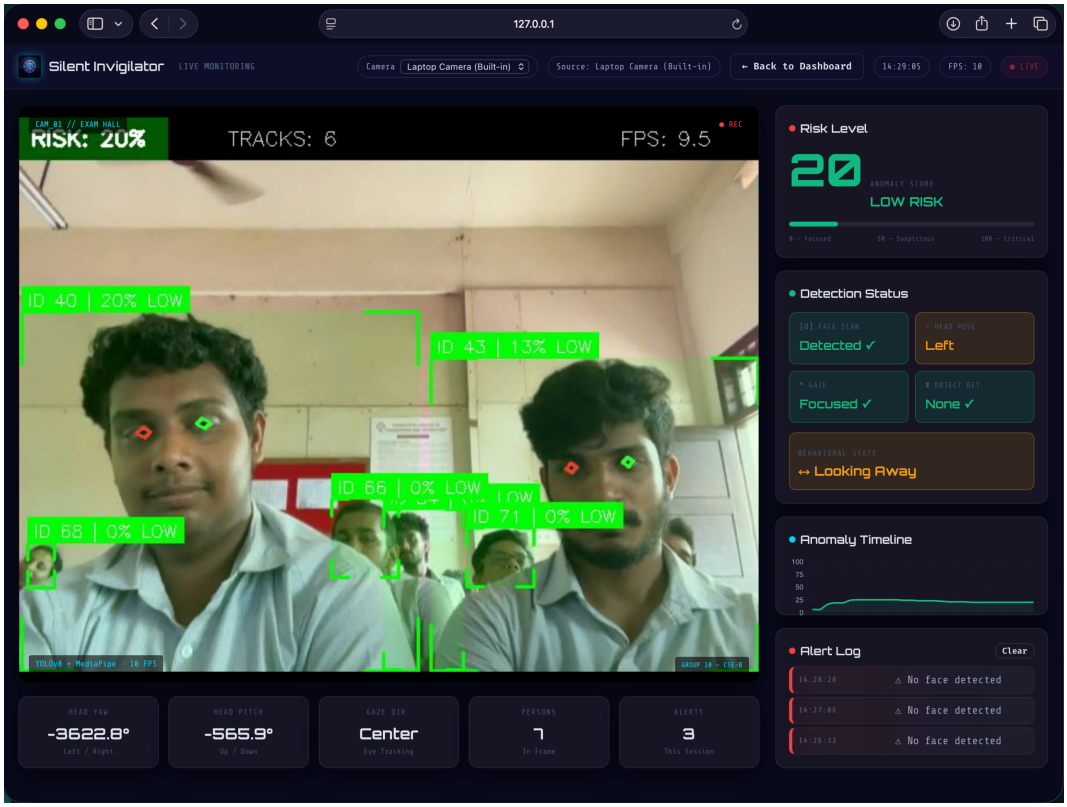

Figure 6.5: Web Dashboard - Real-Time Monitoring UI

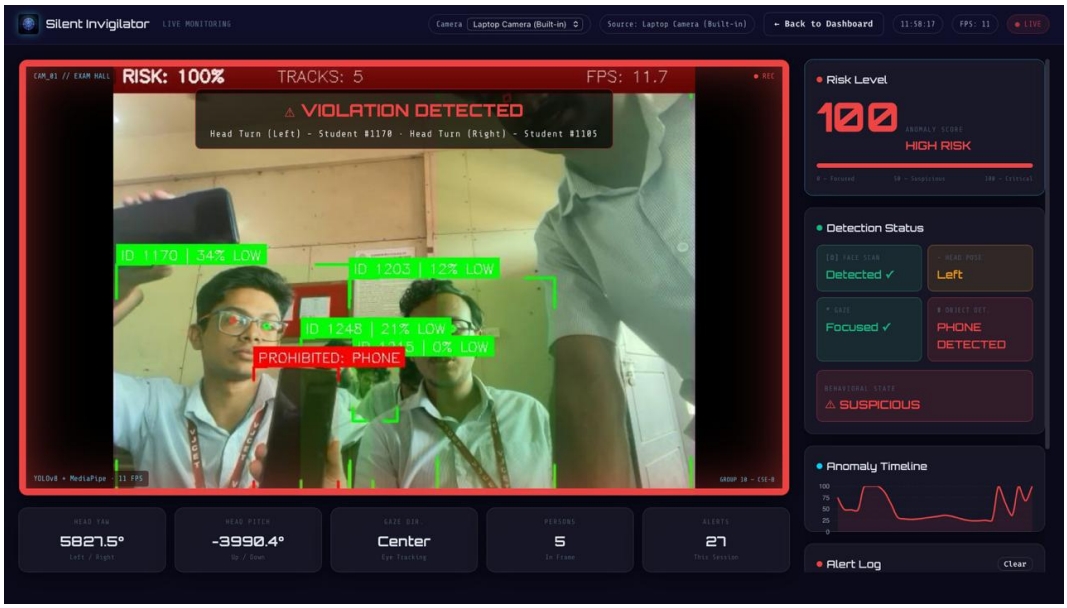

Figure 6.6: Web Dashboard - Real-Time Detection and Alert Management

Mobile Client Interface

The Flutter-based mobile application provides:

- Remote live-feed monitoring screen with real-time risk and detection overlays
- Role-based dashboards for administrator and teacher/invigilator access flows

- API-driven alert and statistics views using backend /api/alerts and /api/stats endpoints
- Demo fallback mode for monitoring UI when backend connectivity is unavailable

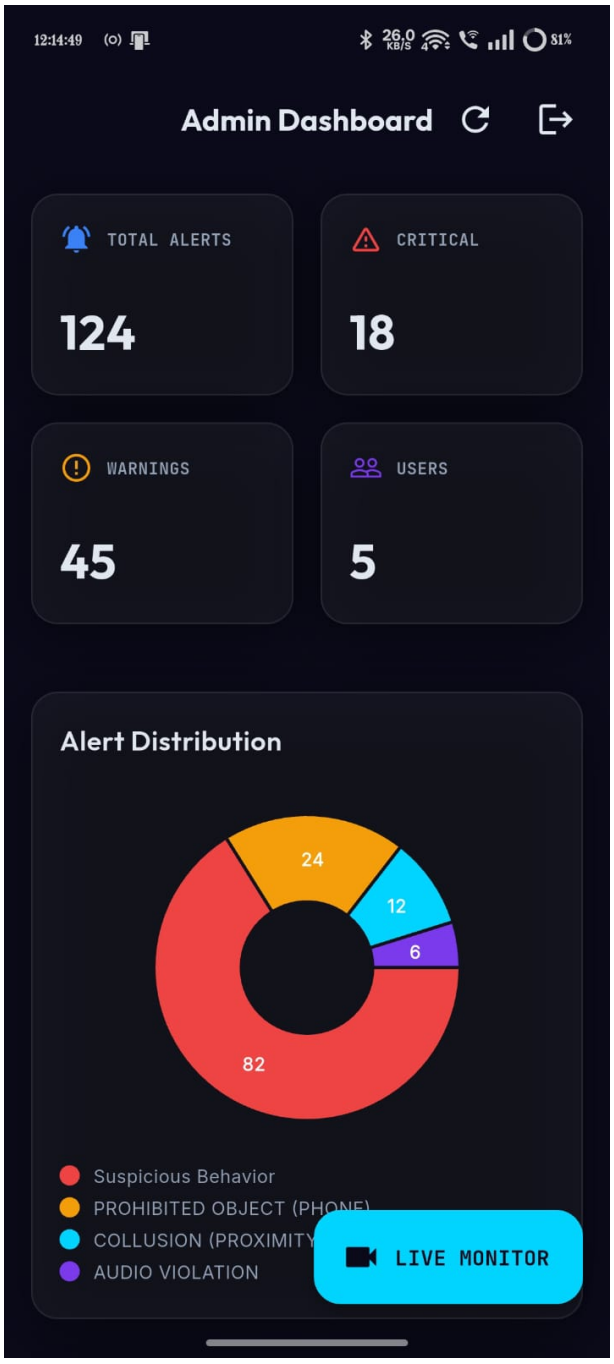

Figure 6.7: Mobile Client - Remote Monitoring Interface

Chapter 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The Silent Invigilator project successfully demonstrates the viability and value of autonomous AI-driven examination invigilation as a complement to or replacement for traditional manual invigilation approaches. Through the integration of advanced computer vision techniques—including 3D head pose estimation via solvePnP, eye gaze direction analysis via iris tracking, and efficient object detection via YOLOv8 with SAHI—the system achieves real-time detection of exam malpractice behaviors with minimal false-positive rates.

Key achievements of this project include:

1. **Robust Multi-Modal Detection**: The system successfully integrates five distinct behavioral modalities into a unified risk-scoring framework, achieving comprehensive yet efficient detection of suspicious behaviors.

2. **Real-Time Performance**: Maintaining 30+ FPS processing on commodity hardware demonstrates the feasibility of real-time AI invigilation within institutional budget constraints.

3. **Privacy-Respecting Architecture**: The system processes only exam-relevant behavioral indicators, avoiding collection of biometric identification data or personal student information beyond session-level anonymized tracking.

4. **Institutional Integration**: Flexible deployment modes (standalone, web, mobile) enable seamless integration with diverse institutional examination workflows.

5. **Comprehensive Audit Trail**: Detailed logging and queryable databases provide objective evidence suitable for academic integrity proceedings and institutional accountability.

6. **Extensible Design**: Modular architecture with clear interfaces enables straightforward addition of new detection modalities or adaptation of thresholds to institution-specific requirements.

The system represents a significant advancement toward reducing reliance on subjective human judgment, minimizing institutional overhead, and creating objective, auditable examination environments. While manual invigilation will likely continue to play a role in academic integrity assurance, The Silent Invigilator provides an enabling technology that can substantially reduce resource requirements while improving detection consistency and reducing human bias.

7.2 Future Scope

Future development opportunities for The Silent Invigilator include:

1. **Advanced Behavior Modeling**: Integration of machine learning models trained on institutional historical data to learn institution-specific baseline behaviors and adapt detection thresholds dynamically.
2. **Multi-Camera Fusion**: Support for multiple camera angles per examination hall with camera fusion techniques enabling 3D scene reconstruction and occlusion handling.
3. **Audio Analysis**: Integration of audio processing for voice activity detection and speech analysis to identify unauthorized communication or dictation.
4. **Biometric Authentication**: Optional integration of fingerprint or facial recognition for automated student identity verification at examination entry.
5. **IoT Device Integration**: Support for wearable sensors (smartwatches, wireless earpieces) detection via electromagnetic field sensing.
6. **Predictive Escalation**: Machine learning-based prediction of examination credibility risk based on entire session patterns, enabling proactive intervention strategies.
7. **Cross-Institutional Intelligence Sharing**: Anonymized aggregation of detection patterns across multiple institutions to identify emerging malpractice techniques.
8. **Mobile Hardening**: Enhanced detection of mobile phone usage patterns including screen reflection recognition and thermal imaging for concealed device detection.
9. **Third-Party Integration**: APIs and plugins enabling integration with learning management systems (Canvas, Blackboard, Moodle) for seamless student registration and result reporting.
10. **Explainable AI**: Development of interpretability layers showing how specific detection decisions were reached, supporting academic integrity proceedings by providing transparent reasoning.

11. **Cross-Platform Deployment**: Support for Linux and macOS systems, as well as cloud-based deployment options for distributed examination centers.

12. **Regulatory Compliance Modules**: Pre-configured compliance modules for specific examination authority regulations (e.g., International Baccalaureate, Cambridge Assessment).

These enhancements, while beyond the current project scope, are architecturally feasible within the modular design framework and represent logical extensions as institutional adoption expands and detection requirements evolve.

References

[1] Duta, I. C., Martinez, B., & Uijlings, J. R. (2020). Detecting defects in SEM images of nanofibrous materials. *IEEE Transactions on Industrial Informatics*, 16(10), 6328–6338.

[2] Redmon, J., & Farhadi, A. (2018). YOLOv3: An incremental improvement. arXiv preprint arXiv:1804.02767.

[3] Ultralytics. (2023). YOLOv8: Cutting Edge Performance Meets Accessibility. Retrieved from https://github.com/ultralytics/ultralytics

[4] Bradski, G., & Kaehler, A. (2008). Learning OpenCV: Computer vision with the OpenCV library. O'Reilly Media.

[5] Lugaresi, C., et al. (2019). MediaPipe: A Framework for Perceiving and Processing Reality. In *3rd IEEE International Conference on Computer Vision Workshop (CVPRW)* (pp. 2496–2504).

[6] Zhou, X., Wang, D., & Krähenbühl, P. (2020). Objects as Points. arXiv preprint arXiv:1904.07850.

[7] Dosovitskiy, A., Fischer, P., Ilg, E., Hausser, P., Cimpoi, C., Dosovitskiy, A., ... & Brox, T. (2015). FlowNet: Learning optical flow with convolutional networks. In *Proceedings of the IEEE international conference on computer vision* (pp. 2440–2448).

[8] Geitgey, A. (2016). face_recognition: The World's Simplest Facial Recognition Library. Retrieved from https://github.com/ageitgey/face_recognition

[9] PyJWT Contributors. (2020). JSON Web Token (JWT) Authentication. Retrieved from https://pyjwt.readthedocs.io/

[10] Wikipedia Contributors. (2023). Perspective-n-Point. Retrieved from https://en.wikipedia.org/wiki/Perspective-n-Point

[11] OpenCV Documentation. (2023). Pose Estimation. Retrieved from https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html

[12] MediaPipe Contributors. (2023). MediaPipe Face Detection. Retrieved from https://google.github.io/mediapipe/solutions/face_detection.html

[13] Stack Overflow Community.(2023). Computer Vision Detection Techniques. Retrieved from https://stackoverflow.com/questions/tagged/computer-vision

[14] TensorFlow Documentation. (2023). Object Detection API. Retrieved from https://github.com/tensorflow/models/tree/master/research/object_detection

[15] Flask Documentation. (2023). Flask by Example. Retrieved from https://flask.palletsprojects.com/

[16] SQLite Documentation. (2023). Query Language. Retrieved from https://www.sqlite.org/lang.html

[17] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

[18] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

[19] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.

[20] Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 2818–2826).

Appendix A

Implementation Sketches - Core Modules

The following listings are representative implementation sketches aligned with the deployed design. Exact production code resides in the project source files under backend and mobile_app modules.

A.1 Head Pose Estimation Implementation Sketch

```
class Stabilizer:
    def __init__(self, method='exponential', alpha=0.2, window_size=30):
        self.alpha = alpha
        self.method = method
        self.window_size = window_size
        self.value = None
        self.history = deque(maxlen=window_size)

    def update(self, value):
        if self.value is None:
            self.value = value
        else:
            if self.method == 'exponential':
                self.value = self.alpha * value + (1 - self.alpha) * self.value
            elif self.method == 'moving_average':
                self.history.append(value)
                self.value = sum(self.history) / len(self.history)
        return self.value


def calculate_head_pose(face_landmarks):
    """
    Calculate 3D head pose (pitch, yaw, roll) from facial landmarks.
    Uses solvePnP with 6 reference points on the face.
    """
    # Define 3D model of face (nose, chin, eyes)
```

```
    model_points = np.array([
        [0.0, 0.0, 0.0],          # Nose tip
        [0.0, -330.0, -65.0],     # Chin
        [-225.0, 170.0, -135.0],  # Left eye
        [225.0, 170.0, -135.0],   # Right eye
        [-150.0, -150.0, -125.0], # Left mouth
        [150.0, -150.0, -125.0]   # Right mouth
    ], dtype="double")

    # Extract 2D image landmarks
    image_points = np.array([
        face_landmarks.landmark[1],      # Nose tip
        face_landmarks.landmark[152],    # Chin
        face_landmarks.landmark[33],     # Left eye
        face_landmarks.landmark[263],    # Right eye
        face_landmarks.landmark[61],     # Left mouth
        face_landmarks.landmark[291]     # Right mouth
    ], dtype="double")

    # Camera matrix (calibrated for 640x480 resolution)
    camera_matrix = np.array([
        [500.0, 0.0, 320.0],
        [0.0, 500.0, 240.0],
        [0.0, 0.0, 1.0]
    ], dtype="double")

    # Solve PnP
    success, rvec, tvec = cv2.solvePnP(
        model_points, image_points, camera_matrix, distCoeffs=None
    )

    if not success:
        return None, None, None

    # Convert rotation vector to rotation matrix
    rotation_matrix, _ = cv2.Rodrigues(rvec)

    # Extract Euler angles
    (x, y, z) = cv2.RQDecomp3x3(rotation_matrix)[0:2]

    return x, y, z


def get_gaze_ratio(face_landmarks):
    """
```

```
    Calculate gaze direction ratio based on iris position.
    Returns normalized ratio indicating gaze left/right position.
    """
    # Iris keypoints (in MediaPipe convention)
    left_eye_iris = face_landmarks.landmark[473]  # Left eye iris
    right_eye_iris = face_landmarks.landmark[468]  # Right eye iris

    # Eye corner keypoints
    left_eye_inner = face_landmarks.landmark[133]  # Left eye inner corner
    left_eye_outer = face_landmarks.landmark[33]   # Left eye outer corner

    right_eye_inner = face_landmarks.landmark[362]  # Right eye inner corner
    right_eye_outer = face_landmarks.landmark[263]  # Right eye outer corner

    # Left eye gaze ratio
    horizontal_iris_left = left_eye_iris.x - left_eye_inner.x
    horizontal_eye_left = left_eye_outer.x - left_eye_inner.x
    ratio_left = horizontal_iris_left / (horizontal_eye_left + 1e-6)

    # Right eye gaze ratio
    horizontal_iris_right = right_eye_iris.x - right_eye_inner.x
    horizontal_eye_right = right_eye_outer.x - right_eye_inner.x
    ratio_right = horizontal_iris_right / (horizontal_eye_right + 1e-6)

    return (ratio_left + ratio_right) / 2.0
```

A.2 Risk Scoring Algorithm Implementation Sketch

```
def compute_student_risk_score(student_record):
    """
    Compute weighted risk score from multiple behavioral indicators.
    Risk score in range [0, 100].
    """
    # Get recent behavioral metrics
    recent_risk_scores = list(student_record['risk_scores'])
    head_poses = list(student_record['head_poses'])
    gaze_directions = list(student_record['gaze_directions'])
    mouth_activities = list(student_record['mouth_activities'])

    if not recent_risk_scores:
        return 0.0

    # Normalize indicators to [0, 1]
    object_confidence = float(student_record.get('object_confidence', 0))
```

```
    # Head deviation: check if pitch/yaw exceeds thresholds
    head_deviation = 0.0
    if head_poses:
        pitch, yaw, roll = head_poses[-1]
        if abs(pitch) > 35 or abs(yaw) > 40:
            head_deviation = 1.0
        elif abs(pitch) > 20 or abs(yaw) > 25:
            head_deviation = 0.5

    # Gaze deviation: check for sustained gaze aversion
    gaze_deviation = 0.0
    if gaze_directions:
        # Count frames with non-forward gaze
        non_forward_count = sum(1 for g in gaze_directions[-10:]
                                if g != 'forward')
        gaze_deviation = min(1.0, non_forward_count / 10.0)

    # Mouth activity
    mouth_activity = 0.0
    if mouth_activities:
        mouth_activity = float(sum(mouth_activities[-10:]) / len(mouth_activities[-10:]))

    # Temporal anomaly score
    anomaly_score = np.var(recent_risk_scores) / 1000.0 if len(recent_risk_scores) > 1 else 0.0
    anomaly_score = min(1.0, anomaly_score)

    # Weighted aggregation (weights sum to 1.0)
    composite_score = (
        0.40 * object_confidence +
        0.22 * gaze_deviation +
        0.16 * head_deviation +
        0.14 * mouth_activity +
        0.08 * anomaly_score
    )

    return int(100 * composite_score)
```

A.3 Backend Flask Service Sketch

```
@app.route('/api/sessions/<session_id>/status', methods=['GET'])
@jwt_required_api
def get_session_status(session_id):
    """Get real-time status of an examination session."""
```

```
    try:
        conn = sqlite3.connect(DB_FILE)
        c = conn.cursor()

        # Fetch session
        c.execute('SELECT * FROM exam_sessions WHERE id = ?',
                  (session_id,))
        session = c.fetchone()
        if not session:
            return jsonify({'error': 'Session not found'}), 404

        # Fetch recent alerts
        c.execute('''SELECT * FROM alerts
                     WHERE session_id = ?
                     ORDER BY timestamp DESC
                     LIMIT 50''',
                  (session_id,))
        alerts = c.fetchall()

        # Fetch student tracking statistics
        c.execute('''SELECT * FROM student_tracking
                     WHERE session_id = ?''',
                  (session_id,))
        students = c.fetchall()

        conn.close()

        return jsonify({
            'session': {
                'id': session[0],
                'name': session[1],
                'start_time': session[3],
                'status': session[5]
            },
            'alerts': [{'id': a[0], 'severity': a[3],
                        'timestamp': a[4]} for a in alerts],
            'student_count': len(students),
            'high_risk_count': sum(1 for s in students if s[4] > 70)
        })
    except Exception as e:
        return jsonify({'error': str(e)}), 500
```

A.4 Database Schema (SQLite)

```
-- Examination sessions table
CREATE TABLE IF NOT EXISTS exam_sessions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    exam_name TEXT NOT NULL,
    class_info TEXT,
    start_time TEXT,
    end_time TEXT,
    status TEXT DEFAULT 'in_progress',
    camera_source TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- Alerts table
CREATE TABLE IF NOT EXISTS alerts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    session_id INTEGER NOT NULL,
    track_id INTEGER,
    severity TEXT,
    risk_score INTEGER,
    timestamp TEXT,
    event_type TEXT,
    description TEXT,
    parameters TEXT,
    acknowledged INTEGER DEFAULT 0,
    notes TEXT,
    FOREIGN KEY(session_id) REFERENCES exam_sessions(id)
);

-- Student tracking table
CREATE TABLE IF NOT EXISTS student_tracking (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    session_id INTEGER NOT NULL,
    track_id INTEGER,
    avg_risk_score REAL,
    max_risk_score INTEGER,
    min_risk_score INTEGER,
    alert_count INTEGER,
    behavior_summary TEXT,
    FOREIGN KEY(session_id) REFERENCES exam_sessions(id)
);
-- Audit logs table
CREATE TABLE IF NOT EXISTS activity_logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    action TEXT,
```

```
    resource TEXT,
    timestamp TEXT DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY(user_id) REFERENCES users(id)
);
```